

**UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA**



**DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE**

**CORSO DI LAUREA IN INGEGNERIA DELL'INFORMAZIONE**

## **Sviluppo software e hardware di un mixer audio digitale**

**Relatore**

Prof. Meneghini Matteo

**Laureando**

De Col Francesco  
Matricola n°2042803

Anno accademico: 2024-2025

Data di laurea: 25/09/2025



# Sommario

Una delle operazioni più comuni nell'ambito dell'elettronica è il filtraggio di segnali, sia analogici che digitali. Esso consente di modificarne alcune caratteristiche come, ad esempio, ampiezza o spettro di frequenza. Nel caso dei segnali digitali, il filtraggio viene eseguito utilizzando microcontrollori o microprocessori, in funzione delle necessità computazionali.

In questo progetto verrà sviluppato un mixer audio digitale, il quale avrà in ingresso una serie di segnali audio analogici da campionare, filtrare e sommare. Questo sistema verrà implementato usando una scheda NUCLEO-G431RB e una serie di circuiti aggiuntivi atti ad interfacciare correttamente i segnali in ingresso al microcontrollore.

Questo elaborato sarà quindi diviso in quattro capitoli:

- **Progettazione:** in questo capitolo, verranno definite le specifiche di progetto, le componenti utilizzate, le motivazioni sulle scelte delle stesse e le loro caratteristiche d'interesse per il sistema.
- **Implementazione:** verrà discusso il processo di realizzazione in dettaglio di ogni parte del sistema.
- **Collaudo:** saranno mostrati e commentati i risultati delle misurazioni effettuate sul sistema in alcune sessioni di laboratorio.
- **Conclusioni:** infine, saranno discussi i risultati delle misurazioni e verranno dati alcuni possibili miglioramenti a questo progetto.



# Indice

|                                                       |           |
|-------------------------------------------------------|-----------|
| Elenco delle figure . . . . .                         | v         |
| Elenco dei listati . . . . .                          | vii       |
| <b>1 Progettazione</b>                                | <b>1</b>  |
| 1.1 Segnali audio e mixer: premessa . . . . .         | 1         |
| 1.2 Specifiche di progetto . . . . .                  | 2         |
| 1.3 Scheda Nucleo . . . . .                           | 2         |
| 1.3.1 Convertitori analogico-digitale (ADC) . . . . . | 3         |
| 1.3.2 Convertitori digitale-analogico (DAC) . . . . . | 8         |
| 1.3.3 Timer (TIM) . . . . .                           | 9         |
| 1.4 Input conditioning . . . . .                      | 9         |
| 1.5 Filtri digitali . . . . .                         | 10        |
| 1.6 STM32CubeIDE . . . . .                            | 12        |
| <b>2 Implementazione</b>                              | <b>13</b> |
| 2.1 Input conditioning . . . . .                      | 13        |
| 2.2 Filtri digitali . . . . .                         | 16        |
| 2.2.1 Filtri passa-basso . . . . .                    | 18        |
| 2.2.2 Filtri passa-banda . . . . .                    | 19        |
| 2.2.3 Filtri passa-alto . . . . .                     | 20        |
| 2.3 Scheda Nucleo . . . . .                           | 23        |
| 2.3.1 Configurazione microcontrollore . . . . .       | 23        |
| 2.3.2 Codice sorgente . . . . .                       | 25        |
| <b>3 Collaudo</b>                                     | <b>33</b> |
| 3.1 Input conditioning . . . . .                      | 34        |
| 3.2 Scheda Nucleo . . . . .                           | 37        |
| <b>4 Conclusioni</b>                                  | <b>45</b> |

|                          |           |
|--------------------------|-----------|
| <b>A Codice sorgente</b> | <b>47</b> |
| <b>Bibliografia</b>      | <b>65</b> |

# Elenco delle figure

|      |                                                                            |    |
|------|----------------------------------------------------------------------------|----|
| 1.1  | Scheda NUCLEO-G431RB[4] . . . . .                                          | 3  |
| 1.2  | Diagramma a blocchi ADC[7] . . . . .                                       | 4  |
| 1.3  | Diagramma a blocchi ADC1[7] . . . . .                                      | 5  |
| 1.4  | Diagramma a blocchi ADC2[7] . . . . .                                      | 6  |
| 1.5  | Diagramma a blocchi DAC[7] . . . . .                                       | 8  |
| 1.6  | Diagramma a blocchi TIM[7] . . . . .                                       | 9  |
| 1.7  | Diagramma Filtri IIR . . . . .                                             | 11 |
| 2.1  | Schema a blocchi progetto . . . . .                                        | 13 |
| 2.2  | Schema elettrico circuito input conditioning . . . . .                     | 14 |
| 2.3  | Risposta in frequenza input conditioning . . . . .                         | 16 |
| 2.4  | Schema a blocchi filtri digitali . . . . .                                 | 16 |
| 2.5  | Risposta in frequenza filtro passa-basso . . . . .                         | 18 |
| 2.6  | Risposta in frequenza filtro passa-banda . . . . .                         | 19 |
| 2.7  | Risposta in frequenza filtro passa-alto a guadagno variabile . . . . .     | 20 |
| 2.8  | Risposta in frequenza filtro passa-alto a coefficienti costanti . . . . .  | 20 |
| 2.9  | Risposta in frequenza filtro equalizzatore . . . . .                       | 21 |
| 3.1  | Circuito input conditioning su breadboard . . . . .                        | 34 |
| 3.2  | Punto operativo DC circuito input conditioning . . . . .                   | 34 |
| 3.3  | Esempio di funzionamento circuito input conditioning (50 Hz) . . . . .     | 35 |
| 3.4  | Confronto risposta in frequenza input conditioning . . . . .               | 36 |
| 3.5  | Misurazione frequenza di campionamento . . . . .                           | 37 |
| 3.6  | Confronto risposta in frequenza filtro passa basso digitale . . . . .      | 38 |
| 3.7  | Confronto risposta in frequenza filtro passa banda digitale . . . . .      | 38 |
| 3.8  | Confronto risposta in frequenza filtro passa alto digitale . . . . .       | 39 |
| 3.9  | Confronto risposta in frequenza filtro passa alto fisso digitale . . . . . | 39 |
| 3.10 | Circuito completo su breadboard . . . . .                                  | 40 |
| 3.11 | Risposta in frequenza filtro equalizzatore . . . . .                       | 41 |

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 3.12 Confronto risposte in frequenza filtro equalizzatore . . . . . | 41 |
| 3.13 FFT canale sinistro . . . . .                                  | 42 |
| 3.14 FFT canale destro . . . . .                                    | 43 |
| 3.15 FFT con segnale a 20 kHz in ingresso . . . . .                 | 43 |



# Elenco dei listati

|      |                                                                    |    |
|------|--------------------------------------------------------------------|----|
| 2.1  | Struct canali audio . . . . .                                      | 25 |
| 2.2  | Definizione valori numerici coefficienti filtri digitali . . . . . | 26 |
| 2.3  | Variabili globali . . . . .                                        | 27 |
| 2.4  | Prototipi funzioni ausiliarie . . . . .                            | 27 |
| 2.5  | Sezione User Code 2 . . . . .                                      | 28 |
| 2.6  | Funzione callback conversione ADC . . . . .                        | 28 |
| 2.7  | Funzione AudioCh_DataUpdate . . . . .                              | 30 |
| 2.8  | Filtro passa-basso digitale . . . . .                              | 31 |
| 2.9  | Filtro passa-banda digitale . . . . .                              | 31 |
| 2.10 | Filtro passa-alto digitale . . . . .                               | 32 |
| 2.11 | Filtro passa-alto digitale a coefficienti fissi . . . . .          | 32 |
| A.1  | File main.c completo . . . . .                                     | 47 |



# Capitolo 1

## Progettazione

Il capitolo che segue ha l'obiettivo di definire le specifiche di progetto, le componenti utilizzate e le motivazioni sulle scelte delle stesse.

### 1.1 Segnali audio e mixer: premessa

Prima di iniziare con la progettazione del mixer, viene fatta una premessa su cosa sono i segnali audio e quali sono le loro caratteristiche principali.

I segnali audio sono segnali analogici che rappresentano uno o un insieme di suoni. Essi possono essere generati, per esempio, da microfoni, strumenti musicali elettrici o da sorgenti elettroniche come un PC o un cellulare. Tipicamente, questi segnali sono rappresentati in campo elettronico da una tensione o da una corrente variabili nel tempo in funzione del suono cui trasmettono. Nel caso dei microfoni, l'onda sonora viene convertita attraverso dei trasduttori in un segnale analogico. Nel caso di uno strumento musicale, come, ad esempio, una chitarra elettrica, il segnale audio viene generato dal movimento o vibrazione di corde o altri corpi. Nel caso di apparecchi digitali, invece, il segnale audio viene generato da appositi generatori di tensione i quali convertono un file audio digitale presente sul dispositivo stesso.

In particolare, i segnali audio sono caratterizzati da una banda di frequenze che interessa l'intervallo 20 Hz - 20kHz circa, il quale coincide circa con la banda di frequenze udibili dagli esseri umani più comunemente accettata[1].

In un mixer audio base, i segnali audio vengono campionati da sorgenti esterne, vengono processati attraverso un numero arbitrario di filtri corrispondenti ad una partizione della banda di frequenze, vengono poi sommati fra loro per ottenere un'unica uscita e sono poi trasmessi a circuiti di amplificazione di potenza o ad altri circuiti elettronici atti a processare ulteriormente i segnali filtrati.

Nel caso di un mixer audio digitale, i segnali audio da mixare vengono convertiti in segnali digitali attraverso dei convertitori analogico-digitale, vengono processati attraverso dei filtri digitali e infine vengono riconvertiti in segnali analogici usando dei convertitori digitale-analogico.

## 1.2 Specifiche di progetto

L'obiettivo di questo progetto è quello di realizzare un mixer audio digitale a due canali stereo.

Il parametro principale per un'applicazione che comprende un'operazione di campionamento di segnali analogici è la frequenza di campionamento: essa, secondo il teorema di Nyquist-Shannon[2], dev'essere pari o superiore al doppio del valore massimo di banda di spettro di frequenze occupata dal segnale da campionare. Se questo vincolo viene rispettato, il segnale campionato può essere interpolato correttamente per ricostruire il segnale analogico di partenza senza perdere informazione utile. Nel caso di un segnale audio, la banda di frequenze interessata è 20Hz - 20kHz: per questo motivo, la frequenza di campionamento  $f_c$  deve essere di almeno 40kHz. In questo progetto, la frequenza di campionamento viene fissata a 40kHz, in modo che i segnali audio vengano campionati correttamente e il tempo a disposizione fra due campionamenti sia il massimo possibile per consentire il corretto svolgimento del filtraggio dei segnali digitali.

Per un mixer audio base, le componenti principali sono:

- tre manopole per canale stereo, usate per gestire il guadagno di tre bande di frequenza: bassa, media ed alta;
- una manopola per canale stereo, usata per attenuare l'ampiezza dei segnali filtrati;
- una manopola generale, usata per attenuare l'ampiezza del segnale audio in uscita dal mixer.

Questo implica l'utilizzo di quattro canali analogici e nove manopole. Saranno quindi necessari 13 ingressi analogici collegati a convertitori analogico-digitale e 2 uscite collegate a convertitori digitale-analogico.

## 1.3 Scheda Nucleo

Le schede Nucleo sono schede programmabili utilizzate principalmente per creare e testare prototipi, offrendo all'utente molte funzionalità software oltre che hardware, mantenendo un valore

economico contenuto. Le periferiche disponibili sulla scheda dipendono sia dal microcontrollore a bordo che dal formato della scheda stessa. Infatti, le schede Nucleo offrono tre formati di circuiti stampati[3].

Per questo progetto viene usata la scheda NUCLEO-G431RB[5], una scheda programmabile che monta un microcontrollore STM32G431RB. Essa presenta diverse caratteristiche fisiche, tra le quali:

- programmatore/debugger ST-LINK a bordo. Esso abilita anche la comunicazione seriale tramite porta COM virtuale (VCP);
- connettori ST Morpho, connettori compatibili con schede di espansione per Arduino UNO e connettore USB Micro-B;
- un pulsante per funzioni definite dall'utente e un pulsante per effettuare il reset del microcontrollore;
- vari jumper per selezionare impostazioni hardware. In particolare, la scheda presenta un jumper con quattro posizioni possibili per selezionare la sorgente della sua tensione di alimentazione.

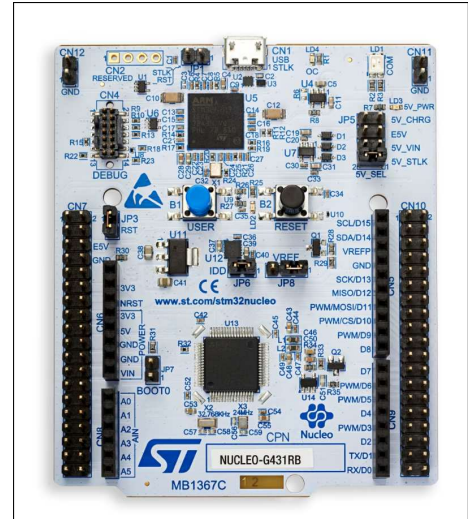


Figura 1.1: Scheda NUCLEO-G431RB[4]

Il microcontrollore STM32G431RB[6] è un microcontrollore con CPU Arm Cortex-M4 a 32 bit con frequenza di clock fino a 170 MHz. Dispone anche di una Floating Point Unit (FPU) per numeri con virgola mobile a precisione singola. Queste due caratteristiche sono il motivo principale per la scelta della scheda: l'alta frequenza di clock risulta necessaria per l'applicazione dei filtri digitali in tempi adeguati all'applicazione mentre la FPU riduce drasticamente il tempo di esecuzione delle operazioni matematiche che coinvolgono numeri con virgola mobile a precisione singola.

Le periferiche principali utilizzate in questo progetto sono i convertitori analogico-digitale (ADC), i convertitori digitale-analogico (DAC) e i timer (TIM). Di seguito viene offerta una breve panoramica su ognuna delle suddette periferiche.

### 1.3.1 Convertitori analogico-digitale (ADC)

Il microcontrollore STM32G431RB monta due convertitori analogico-digitali ad approssimazioni successive. In questo progetto, essi sono utilizzati per convertire il valore di tensione in

ingresso ai loro canali in un valore numerico proporzionale. La tensione in ingresso può variare, al massimo, fra 0 V e 3.3 V. Gli ADC hanno una risoluzione massima di 12 bit (eventualmente riducibile) e hanno un tasso di conversione massimo di 4 milioni di campioni per secondo a risoluzione massima. Nella figura (1.2) è riportato il diagramma a blocchi degli ADC.

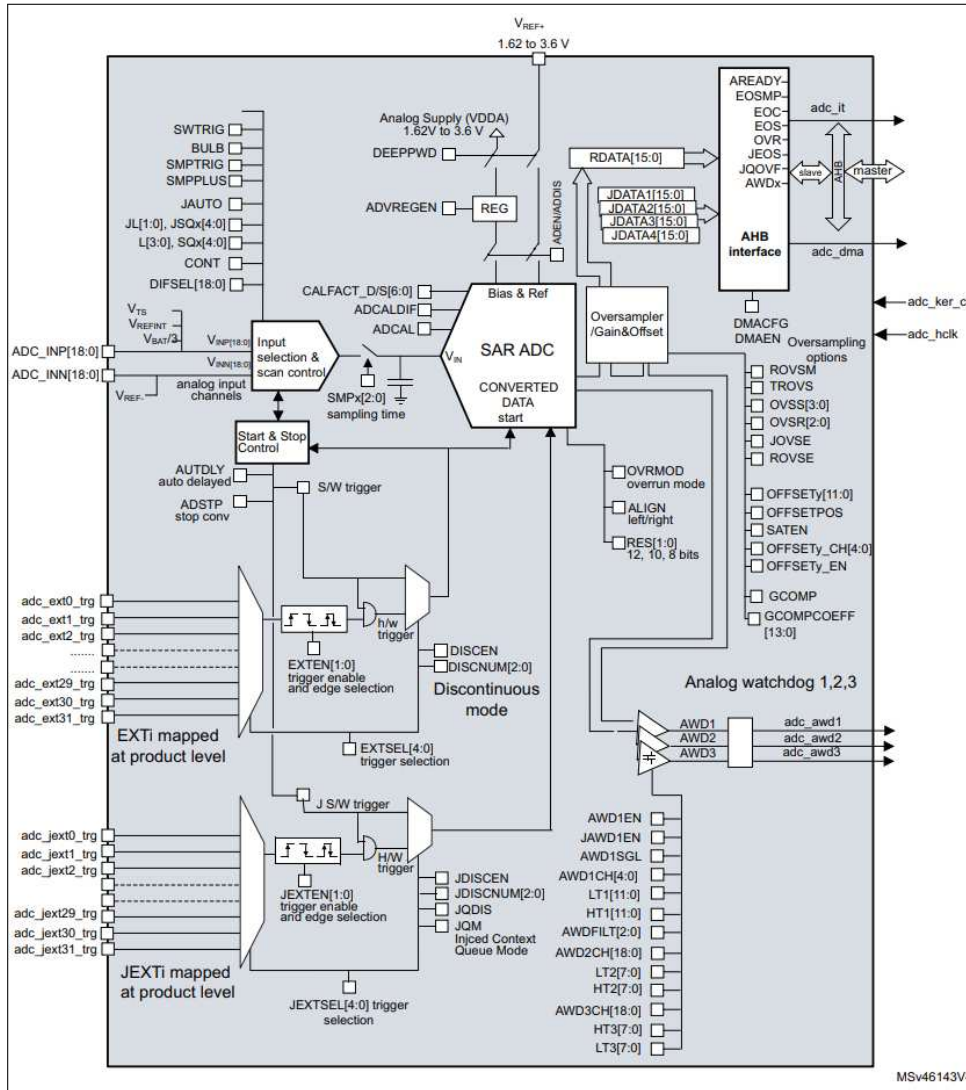


Figura 1.2: Diagramma a blocchi ADC[7]

L'architettura degli ADC presenta canali in comune fra i due convertitori: nelle figure (1.3) e (1.4) sono riportati i diagrammi a blocchi dei canali degli ADC.

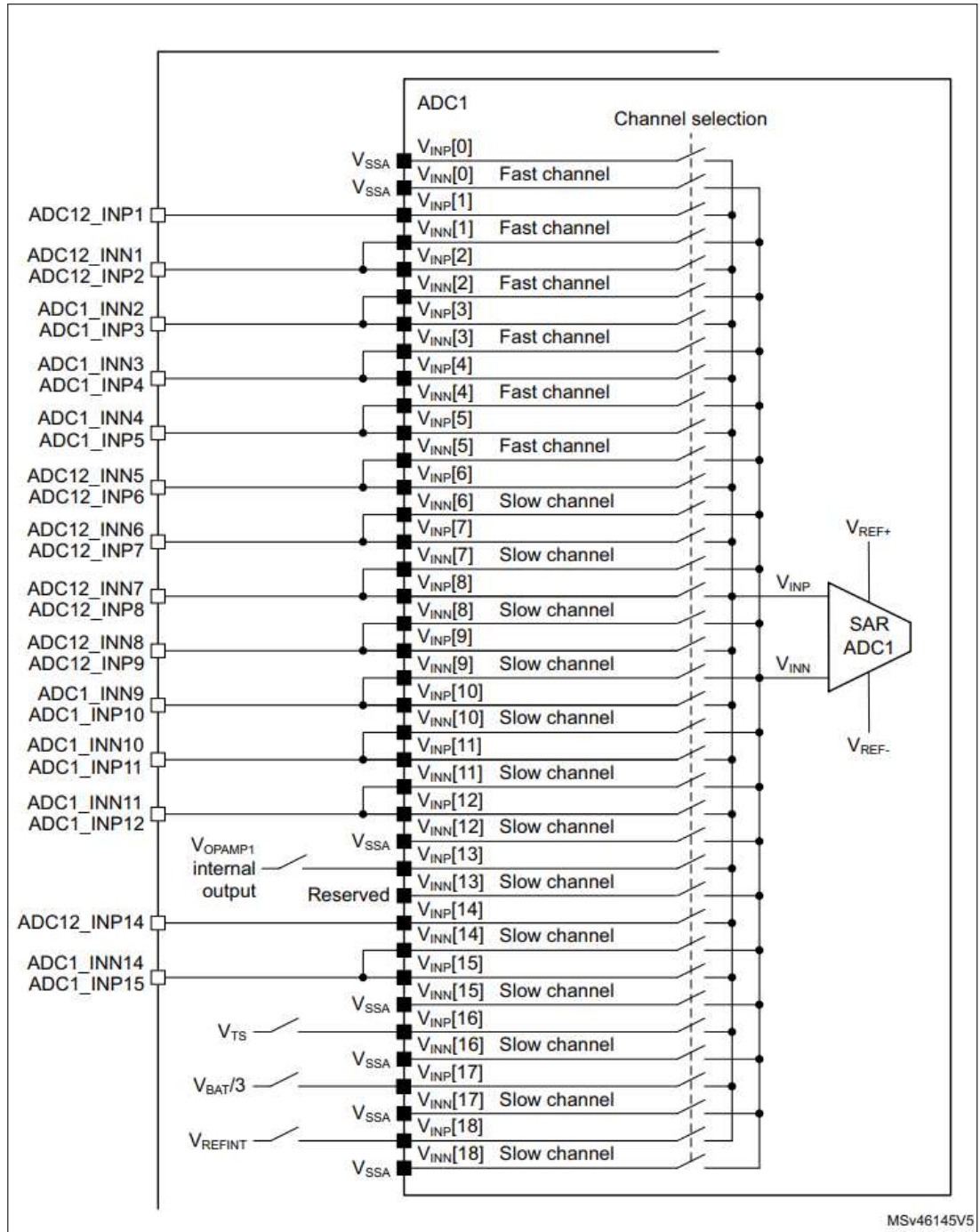


Figura 1.3: Diagramma a blocchi ADC1[7]

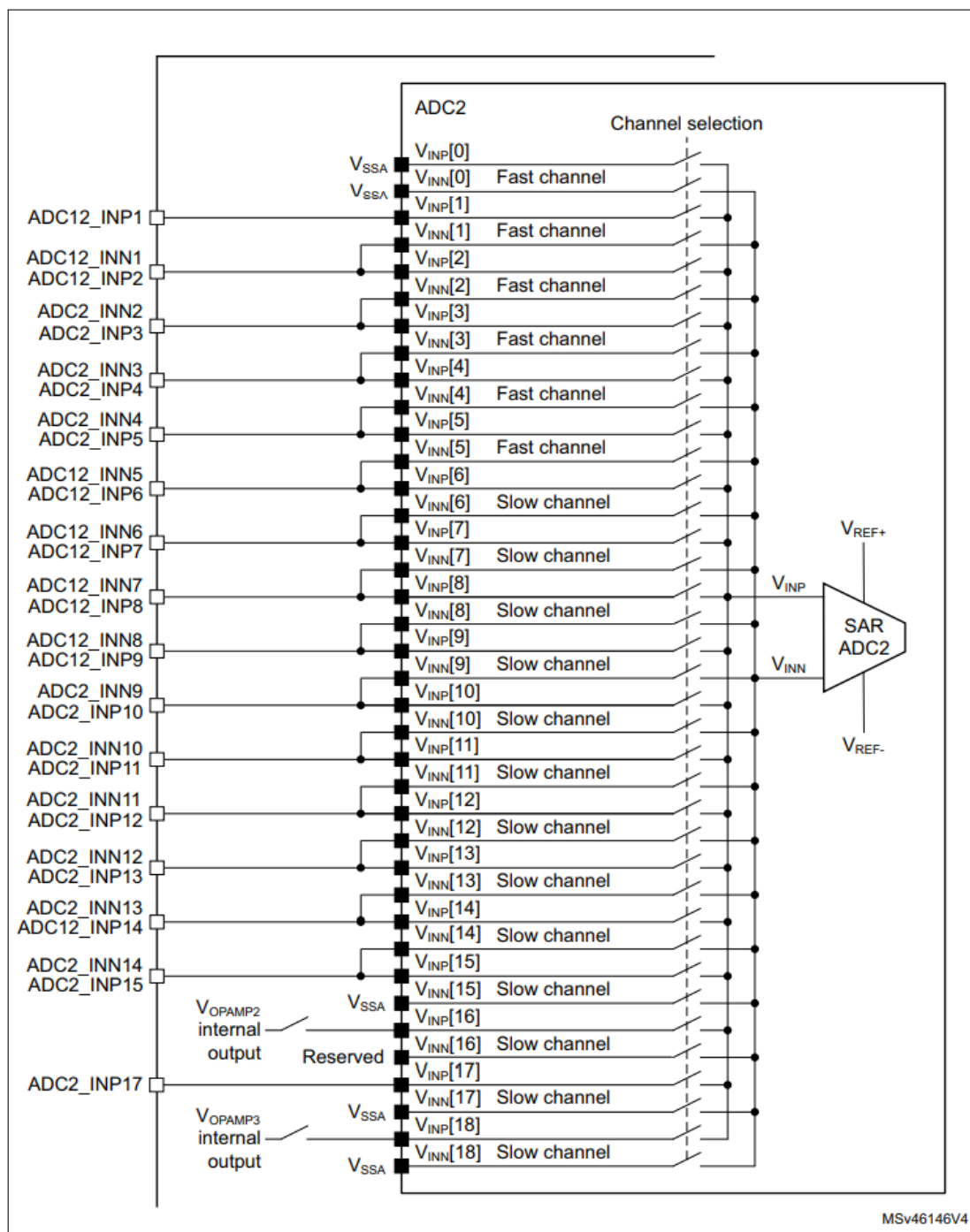


Figura 1.4: Diagramma a blocchi ADC2[7]



Un'altra particolarità di questi due ADC è la possibilità di utilizzarli in modalità duale: essa prevede una configurazione tale per cui ADC1 agisce come master e gestisce i segnali di inizio e fine conversione per entrambi i convertitori. Il risultato delle conversioni degli ADC viene quindi salvato in un registro a 32 bit dove i primi 16 bit corrispondono alla conversione dell'ADC1 e i restanti 16 bit a quella dell'ADC2.

Il segnale d'inizio conversione degli ADC viene generato attraverso l'Update Event di un timer.

### 1.3.2 Convertitori digitale-analogico (DAC)

Il microcontrollore monta due convertitori digitale-analogico, utilizzati per convertire un valore numerico in una tensione proporzionale ad esso. Essi hanno risoluzione a 8 o a 12 bit e i valori da convertire possono essere allineati a sinistra o a destra. Il primo DAC (DAC1) presenta due canali con buffer in output opzionali, mentre il secondo (DAC3) presenta due canali collegabili solo alle periferiche on board del microcontrollore: in questo progetto, viene scelto di utilizzare solo DAC1. In figura (1.5) viene riportato il diagramma a blocchi del DAC.

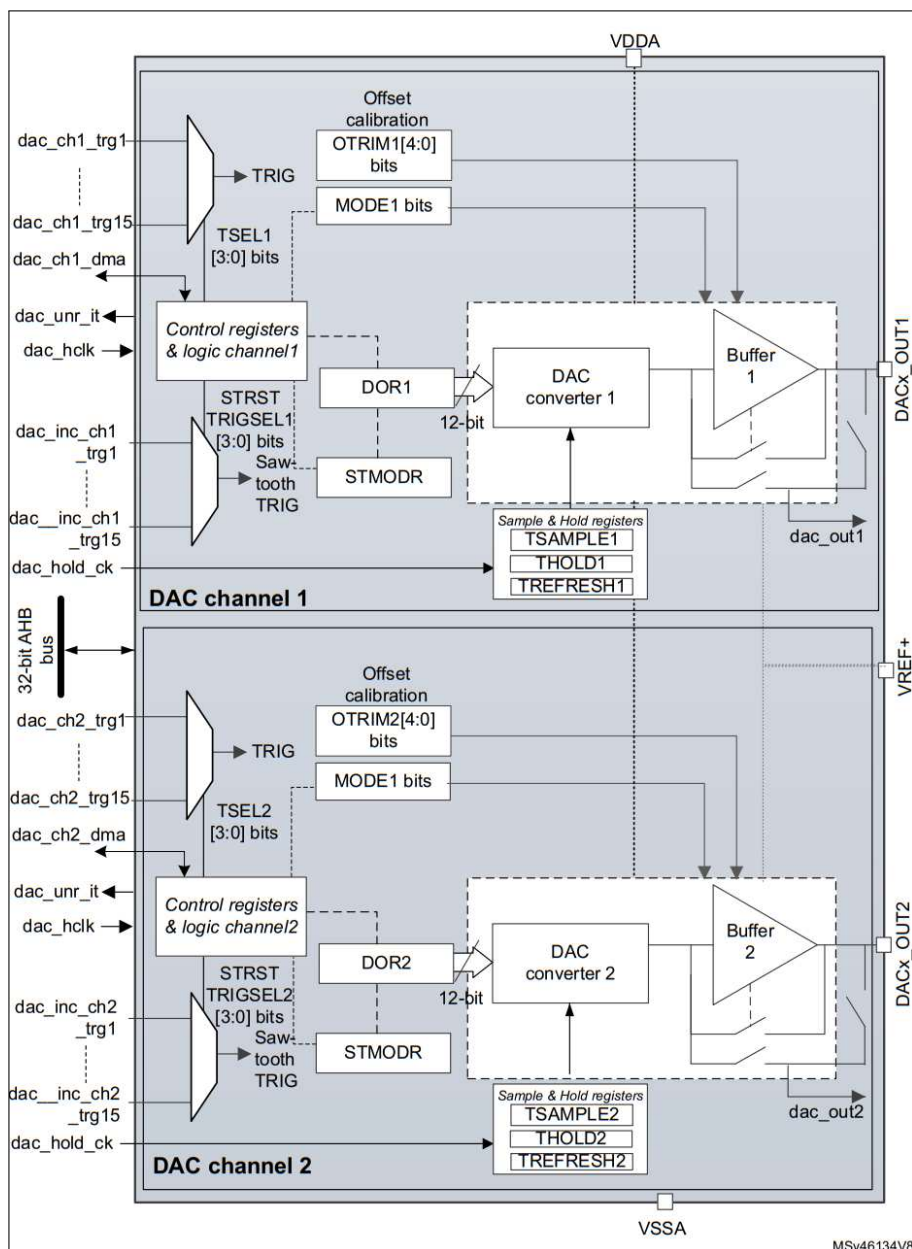


Figura 1.5: Diagramma a blocchi DAC[7]

Il DAC usato può essere utilizzato in modalità duale, in modo da poter pilotare contempo-

ranamente i due canali a disposizione. Inoltre, l'Update Event dello stesso timer usato per gli ADC può essere usato come trigger d'inizio conversione per il DAC.

### 1.3.3 Timer (TIM)

Il microcontrollore monta una serie di timer a scopo e risoluzione variabile. Per questo progetto, viene usato il TIM6, un timer basilare che integra un contatore a ricarica automatica ed un prescaler. Esso verrà usato per generare un Update Event per far iniziare le conversioni ADC con frequenza adeguata alla specifiche di progetto. Nella figura (1.6) è riportato il diagramma a blocchi del timer.

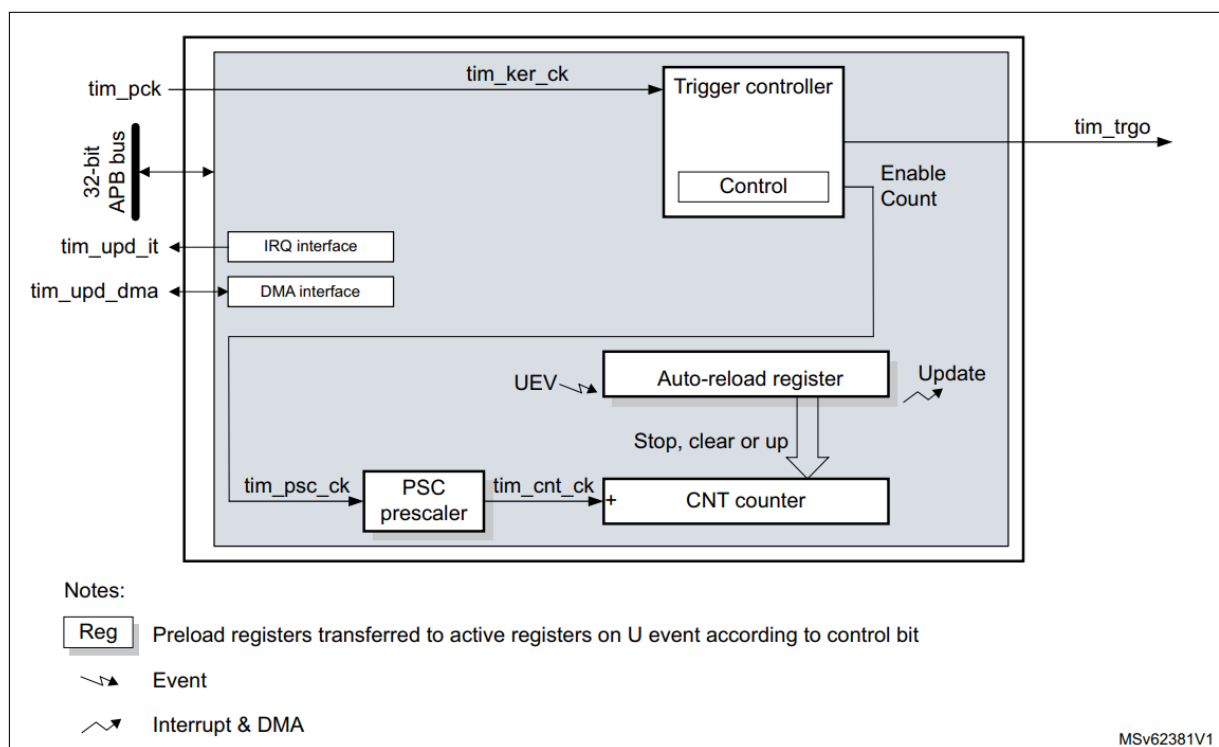


Figura 1.6: Diagramma a blocchi TIM[7]

## 1.4 Input conditioning

Per interfacciare correttamente i segnali audio al microcontrollore è necessario sviluppare un circuito di input conditioning: questo perché, per loro natura, essi sono caratterizzati da uno spettro di frequenze che non presenta una componente costante. L'ampiezza dei segnali varia quindi in un intervallo di tensione attorno a 0 V, assumendo valori di tensione a volte positivi, a volte negativi. Per questo motivo, i segnali audio non possono essere interfacciati direttamente con il convertitore analogico-digitale del microcontrollore. Inoltre, i segnali audio provenien-

ti dall'esterno presentano ampiezza limitata: per questo progetto, si suppone che l'ampiezza massima raggiunta dai segnali audio sia pari a 0.4 V.

Il circuito di input conditioning deve quindi amplificare ed applicare un offset DC al segnale audio, in modo tale da sfruttare l'intervallo di tensione supportato dal convertitore analogico-digitale del microcontrollore.

Per sviluppare il circuito, verrà usato il circuito integrato MCP6002[8], sviluppato da Microchip Technology Inc.: esso integra due amplificatori operazionali per uso generale. In particolare, questi amplificatori presentano uscite rail-to-rail, consentendo alle stesse di poter raggiungere i valori di tensione di alimentazione del circuito. Nella tabella (1.1) sono riportati i parametri d'interesse per questo progetto.

| Parametro                                                      | Minimo | Tipico    | Massimo | Unità      |
|----------------------------------------------------------------|--------|-----------|---------|------------|
| Tensione di alimentazione                                      | 1.8    | -         | 6.0     | V          |
| Slew Rate                                                      | -      | 0.6       | -       | V/ $\mu$ s |
| Massimo intervallo di tensione dell'uscita ( $V_{CC} = 3.3$ V) | 0.025  | -         | 3.275   | V          |
| Corrente di bias in ingresso                                   | -      | $\pm 1.0$ | -       | pA         |

Tabella 1.1: Valori d'interesse MCP6002

## 1.5 Filtri digitali

Per filtrare i segnali audio campionati, vengono utilizzati dei filtri digitali IIR (Infinite Impulse Response)[9]. Essi sono dei filtri caratterizzati da una risposta impulsiva infinita, ovvero che, per intervalli di tempo tendenti ad infinito, essa tende ad un valore non nullo. Un'altra caratteristica di questo tipo di filtri è la dipendenza del valore di uscita dai suoi valori precedenti, oltre che dall'ingresso e dai suoi valori precedenti. Vengono scelti dei filtri digitali del secondo ordine: questo per ottenere una divisione più netta tra le bande di frequenza rispetto a dei filtri del primo ordine.

I filtri IIR sono scelti principalmente per la loro facilità di implementazione e dalla loro efficienza computazionale, essendo il numero di operazioni matematiche limitato (due somme, due sottrazioni e sei moltiplicazioni nel caso di un filtro IIR del secondo ordine). Una considerazione particolare dev'essere data però alla stabilità del filtro, la quale non è garantita in generale.

L'obiettivo sarà di ottenere, inizialmente, una risposta in frequenza complessiva con guadagno unitario nella banda di frequenze interessata dai segnali audio. La risposta in frequenza verrà poi modificata in funzione della posizione dei potenziometri interessati all'attenuazione o all'amplificazione delle rispettive bande di frequenza.

Si è quindi interessati alla definizione generale della funzione di trasferimento per un filtro IIR del secondo ordine.

Dato un filtro digitale del secondo ordine con ingresso un segnale in ingresso  $x[n]$  e un segnale in uscita  $y[n]$ , la sua funzione di trasferimento è definita come nell'equazione (1.1):

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{a_0 + a_1 z^{-1} + a_2 z^{-2}} \quad (1.1)$$

dove i coefficienti  $a_0 \neq 0, a_1, a_2, b_0, b_1, b_2$  sono numeri reali che pesano i campioni dei segnali di ingresso ed uscita in base alla classificazione del filtro.

Sfruttando le proprietà della trasformata zeta[10], si ottiene la seguente equazione alle differenze:

$$y[n] = \frac{1}{a_0} (b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2]) \quad (1.2)$$

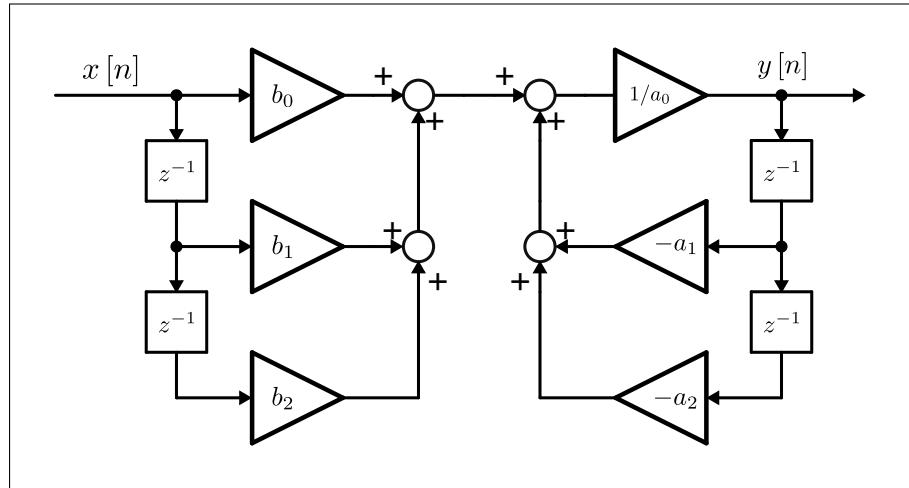


Figura 1.7: Diagramma Filtri IIR

La formula (1.2) verrà quindi utilizzata per definire tre categorie di filtri: passa-basso, passa-banda e passa-alto.

Per calcolare il valore dei coefficienti di ogni filtro si partirà dalla definizione di una funzione di trasferimento di un filtro analogico, per poi applicare alla stessa la trasformata bilineare[11]. Essa viene usata per trasformare una funzione di trasferimento analogico nella sua controparte digitale applicando la seguente approssimazione al filtro analogico.

$$s \leftarrow \frac{2}{T_c} \frac{1 - z^{-1}}{1 + z^{-1}} \quad (1.3)$$

dove  $T_c = 1/f_c$ .

In questo modo, la stabilità del filtro analogico viene preservata e la risposta in frequenza del filtro digitale approssima adeguatamente la risposta in frequenza del filtro analogico di partenza. La trasformata, infatti, introduce una deformazione non lineare della risposta in frequenza particolarmente apprezzabile in alta frequenza: in questo progetto, essa risulta essere trascurabile rispetto alle frequenze di taglio implementate dai filtri.

## 1.6 STM32CubeIDE

STM32CubeIDE[12] è un ambiente di sviluppo integrato C/C++, specifico per i microcontrollori e microprocessori STM32. Esso incorpora le funzioni di configurazione dei microcontrollori, l'editor di codice e le funzioni di debug in un unico software. Inoltre, l'IDE è basata sul framework Eclipse, usa la toolchain GCC per la compilazione del codice sorgente e usa GDB per le sue funzioni di debug.

In questo progetto, STM32CubeIDE verrà utilizzata per:

- configurare il microcontrollore della scheda NUCLEO e le sue periferiche;
- generare, modificare e compilare il codice sorgente del programma;
- effettuare il debug del codice caricato sulla scheda.

# Capitolo 2

## Implementazione

Nel capitolo che segue, verrà illustrata l'implementazione hardware e software del progetto. Nella figura (2.1) viene riportato lo schema a blocchi generale del mixer, comprensivo delle tensioni di alimentazione dei circuiti.

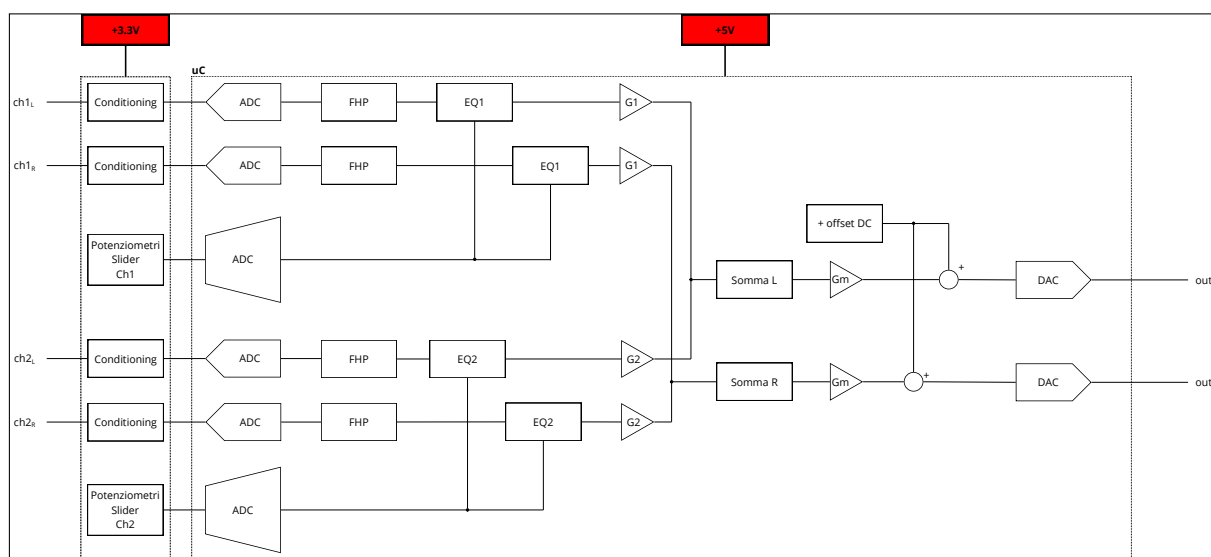


Figura 2.1: Schema a blocchi progetto

### 2.1 Input conditioning

Il circuito di input conditioning deve essere progettato in modo che applichi un offset DC ed amplifichi un segnale audio in ingresso. Il segnale filtrato dovrà quindi essere interfacciato con il convertitore analogico-digitale del microcontrollore: la progettazione del circuito dovrà quindi tenere in considerazione i limiti massimi e minimi di tensione in ingresso al convertitore.

Per lo sviluppo hardware del circuito di input conditioning viene scelto un circuito integrato MCP6002: esso è composto da due amplificatori operazionali rail-to-rail. In questo modo,

alimentando gli amplificatori operazionali con una tensione di 3.3 V, il segnale in uscita al circuito di input conditioning rispetta i limiti di tensione del convertitore analogico-digitale del microcontrollore.

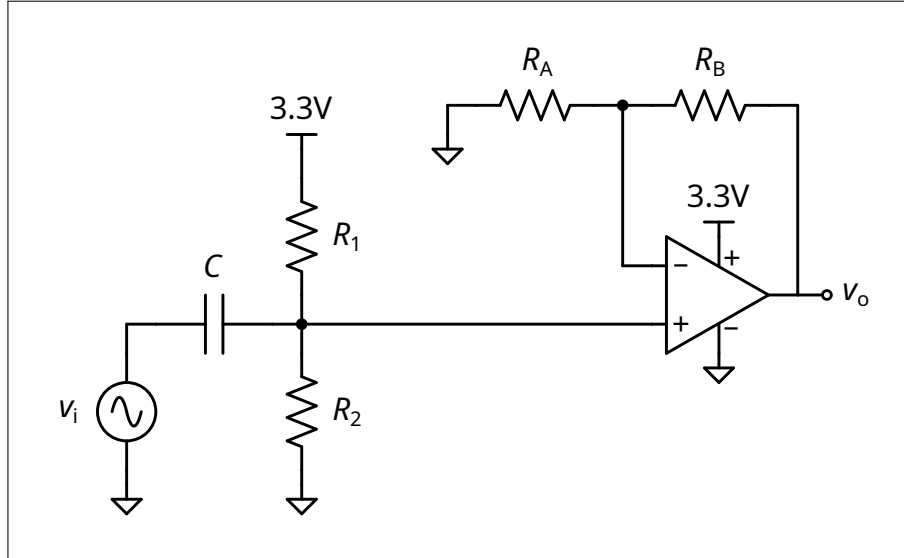


Figura 2.2: Schema elettrico circuito input conditioning

Viene scelta una configurazione non invertente per gli amplificatori operazionali. Inoltre, viene aggiunta una rete di due resistenze ed un condensatore all'ingresso non invertente degli amplificatori atta ad applicare un offset DC e ad agire come un filtro passa-alto per il segnale audio. Il condensatore viene aggiunto anche per proteggere la sorgente audio dalla tensione costante generata dal partitore resistivo.

Per ricavare il valore di guadagno del circuito bisogna determinare il valore delle due resistenze della configurazione non invertente. La formula di calcolo per il guadagno del circuito è data dalla seguente formula:

$$G = 1 + \frac{R_B}{R_A} \quad (2.1)$$

Idealmente, il segnale audio filtrato dovrebbe presentare una componente DC pari a 1.65 V, ovvero metà dell'intervallo di tensione accettato dal convertitore A/D. Questo implica che l'ampiezza massima del segnale amplificato dev'essere pari a 1.65 V: in questo modo, la sovrapposizione fra il segnale audio e l'offset DC può variare fra 0 V e 3.3 V. Ricordando che l'ampiezza massima del segnale audio in ingresso è supposta al massimo pari a 0.4 V, il guadagno massimo del circuito viene dato dal rapporto  $G_{max} = 1.65 \text{ V} / 0.4 \text{ V} = 4.125$ . Usando il valore di guadagno appena calcolato nella formula (2.1), si evince che, al massimo,  $\frac{R_B}{R_A} = 3.125$ . Una coppia di valori delle due resistenze serie E12 che approssimano il valore di guadagno richiesto sono  $R_B = 82 \text{ k}\Omega$  e  $R_A = 27 \text{ k}\Omega$ : il guadagno implementato del circuito è circa pari a 4.037. Questa coppia di resistenze viene scelta anche considerando un eventuale scostamento dell'offset DC



dato dalla scelta delle resistenze nella rete resistiva in ingresso al circuito di input conditioning, in modo da non eccedere i limiti di tensione del convertitore A/D.

Il passo successivo è calcolare il valore delle due resistenze  $R_1$  ed  $R_2$ , usate per generare l'offset DC. Osservando la configurazione del circuito nel suo punto di operazione DC, la tensione applicata all'ingresso non invertente dell'amplificatore operazionale è data dal partitore di tensione dato da  $R_1$  ed  $R_2$ , la quale viene poi amplificata. La tensione in uscita dell'amplificatore operazionale nel suo punto di operazione DC è data è calcolata come di seguito:

$$v_o = 4.037 \frac{R_2}{R_1 + R_2} V_{CC} \quad (2.2)$$

dove  $V_{CC} = 3.3$  V è la tensione di alimentazione del circuito.

Idealmente,  $v_o$  dovrebbe essere pari a 1.65 V: con questa considerazione, il valore di  $R_1$  ed  $R_2$  può essere ricavato. I valori delle due resistenze serie E12 che approssimano adeguatamente il valore della tensione in uscita dell'amplificatore operazionale sono  $R_1 = 330$  k $\Omega$  e  $R_2 = 47$  k $\Omega$ . Si conclude che, con i valori delle resistenze calcolati,  $v_o \simeq 1.661$  V.

Infine, viene calcolato il valore del condensatore. In questo caso, si vuole osservare la rete in ingresso all'amplificatore nel suo punto di operazione AC: il risultato è una rete RC serie dove il valore di resistenza è dato dal parallelo delle resistenze  $R_1$  ed  $R_2$ . Essa agisce come un filtro passa-alto per il segnale audio in ingresso avente frequenza di taglio del filtro pari a:

$$f = \frac{1}{2\pi R_{in} C} \quad (2.3)$$

dove  $R_{in} = R_1 // R_2$ .

La frequenza di taglio viene arbitrariamente fissata a 1Hz: in questo modo, la banda di frequenze interessata dal segnale audio viene filtrata correttamente. Il valore di condensatore che approssima adeguatamente la frequenza di taglio del filtro è  $C = 3.9$   $\mu$ F.

Con i valori di resistenze e condensatore scelti, il segnale audio filtrato può variare in un intervallo di tensione fra 46.2 mV e 3.276 V. Esso rispetta i vincoli sull'intervallo di tensione accettato dal convertitore A/D del microcontrollore.

La risposta in frequenza del circuito completo può essere ricavata usando LTSpice: nella figura (2.3) viene riportato il confronto fra la risposta in frequenza reale (in rosso) e ideale (in verde).

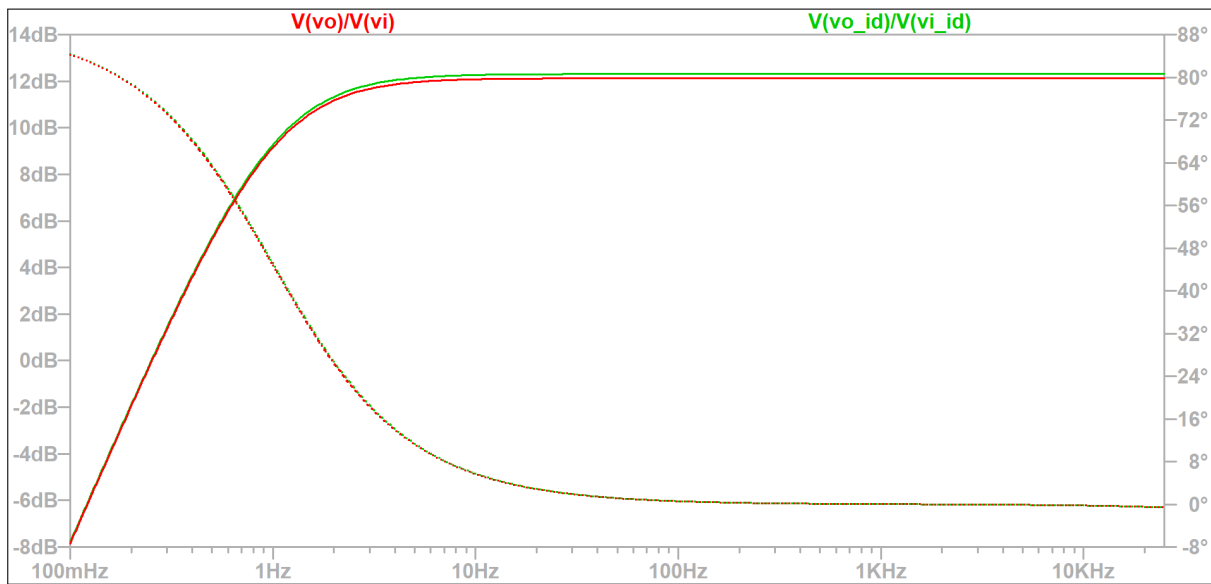


Figura 2.3: Risposta in frequenza input conditioning

## 2.2 Filtri digitali

Ottenuti i campioni digitali provenienti dal convertitore A/D, essi vengono filtrati dal microcontrollore tramite dei filtri IIR del secondo ordine. Per definire i loro parametri, vengono considerati dei filtri analogici del secondo ordine definiti in forma canonica in funzione della banda di frequenze filtrata. Successivamente, verrà applicata la trasformata bilineare per ottenere delle equazioni alle differenze da implementare sul microcontrollore.

Per consentire un filtraggio corretto dei segnali audio, si considera un insieme di filtri come in figura (2.4).

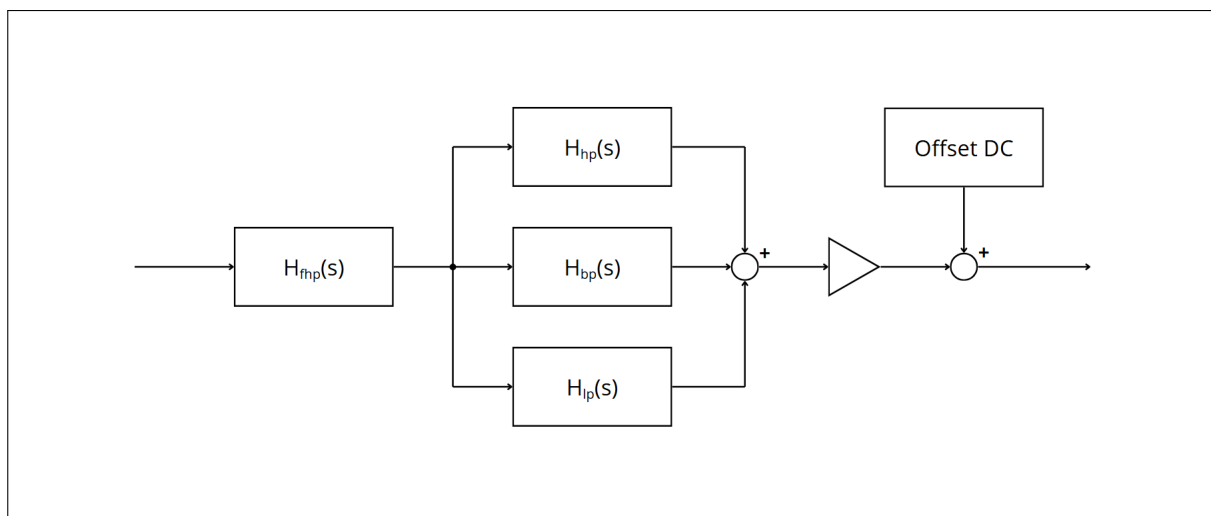


Figura 2.4: Schema a blocchi filtri digitali

Esso può essere diviso in quattro stadi:

- filtro passa-alto a coefficienti costanti: esso riceve in ingresso i campioni provenienti dal convertitore A/D e ne rimuove l'offset DC introdotto dal circuito di input conditioning;
- parallelo di filtro passa-basso, passa-banda e passa-alto: essi filtrano il segnale audio digitale nelle rispettive bande di frequenza. I segnali filtrati vengono poi sommati;
- attenuatore: esso moltiplica il segnale filtrato per un valore compreso fra 0 ed 1;
- offset DC: esso dev'essere introdotto per interfacciare correttamente il risultato del filtraggio del segnale audio al convertitore D/A del microcontrollore.

Nella definizione delle funzioni di trasferimento analogiche, si sceglie di creare dei filtri aventi poli coincidenti e giacenti nel piano reale negativo: in questo modo, i filtri saranno stabili.

Si procede quindi ricavando le formule per il calcolo dei coefficienti delle equazioni alle differenze. Saranno anche riportati i diagrammi di Bode delle funzioni di trasferimento generati usando MATLAB.

### 2.2.1 Filtri passa-basso

Viene considerato un filtro passa-basso analogico del secondo ordine nella seguente forma canonica:

$$W_{lp}(s) = G_{lp} \frac{\omega_{lp}^2}{s^2 + \frac{\omega_{lp}}{Q_{lp}}s + \omega_{lp}^2} \quad (2.4)$$

dove  $\omega_{lp} = 2\pi f_{lp}$  è la sua pulsazione naturale,  $f_{lp} := 800$  Hz è la sua frequenza di taglio,  $Q_{lp} := 0.5$  è il suo fattore di merito e  $G_{lp} = 1$  è il suo guadagno iniziale, modificabile attraverso il rispettivo potenziometro.

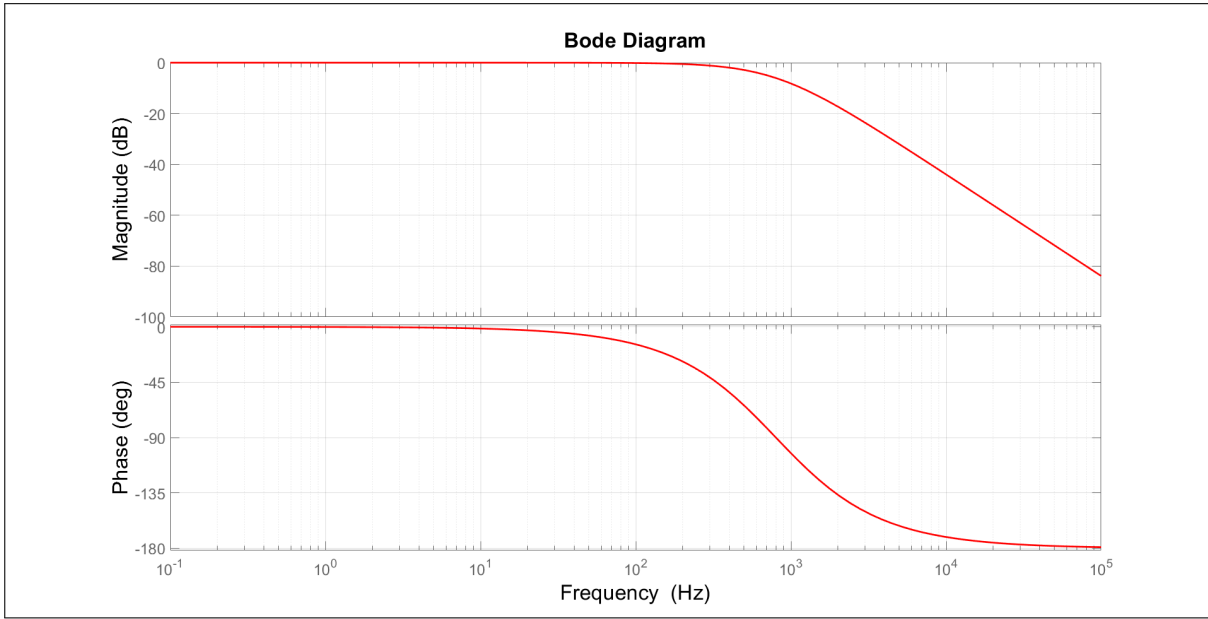


Figura 2.5: Risposta in frequenza filtro passa-basso

Applicando la trasformata bilineare a  $W_{lp}(s)$ , si ottiene la seguente funzione di trasferimento digitale:

$$H_{lp}(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{A_0 + A_1 z^{-1} + A_2 z^{-2}} \quad (2.5)$$

dove:  $A_0 = 4 + 2T_c \frac{\omega_{lp}}{Q_{lp}} + \omega_{lp}^2 T_c^2$ ,  $A_1 = 2\omega_{lp}^2 T_c^2 - 8$ ,  $A_2 = 4 - 2T_c \frac{\omega_{lp}}{Q_{lp}} + \omega_{lp}^2 T_c^2$ ,  $b_0 = b_2 = G_{lp} \omega_{lp}^2 T_c^2$ ,  $b_1 = 2G_{lp} \omega_{lp}^2 T_c^2$ .

## 2.2.2 Filtri passa-banda

Viene considerato un filtro passa-banda analogico del secondo ordine nella seguente forma canonica:

$$W_{bp}(s) = G_{bp} \frac{\frac{\omega_{bp}}{Q_{bp}} s}{s^2 + \frac{\omega_{bp}}{Q_{bp}} s + \omega_{bp}^2} \quad (2.6)$$

dove  $\omega_{bp} = 2\pi f_{bp}$  è la sua pulsazione naturale,  $f_{bp} := 900$  Hz è la sua frequenza di taglio,  $f_B := 1800$  Hz è la sua larghezza di banda passante,  $Q_{bp} := \frac{f_{bp}}{f_B} = 0.5$  è il suo fattore di merito e  $G_{bp} := 1.1$  è il suo guadagno iniziale, modificabile attraverso il rispettivo potenziometro.

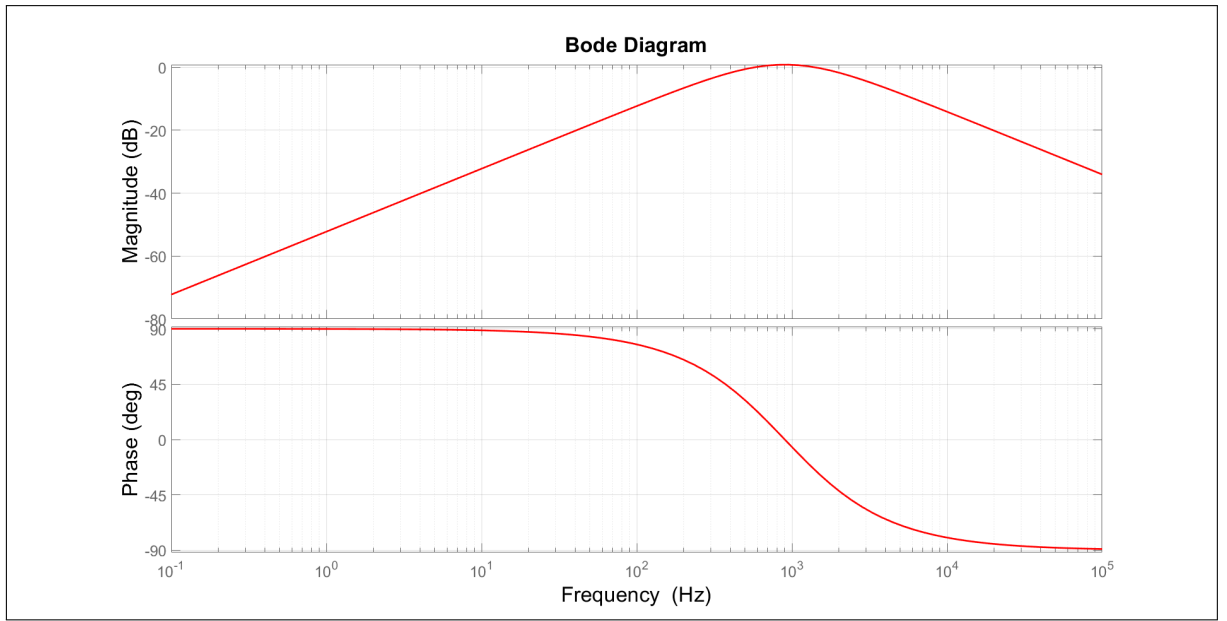


Figura 2.6: Risposta in frequenza filtro passa-banda

Applicando la trasformata bilineare a  $W_{bp}(s)$ , si ottiene la seguente funzione di trasferimento digitale:

$$H_{bp}(z) = \frac{d_0 + d_2 z^{-2}}{C_0 + C_1 z^{-1} + C_2 z^{-2}} \quad (2.7)$$

dove:  $C_0 = 4 + 2T_c \frac{\omega_{bp}}{Q_{bp}} + \omega_{bp}^2 T_c^2$ ,  $C_1 = 2\omega_{bp}^2 T_c^2 - 8$ ,  $C_2 = 4 - 2T_c \frac{\omega_{bp}}{Q_{bp}} + \omega_{bp}^2 T_c^2$ ,  $d_0 = 2T_c G_{bp} \frac{\omega_{bp}}{Q_{bp}}$ ,  $d_2 = -d_0$ .

### 2.2.3 Filtri passa-alto

Viene considerato un filtro passa-alto analogico del secondo ordine nella seguente forma canonica:

$$W_{hp}(s) = G_{hp} \frac{s^2}{s^2 + \frac{\omega_{hp}}{Q_{hp}} s + \omega_{hp}^2} \quad (2.8)$$

dove  $\omega_{hp} = 2\pi f_{hp}$  è la sua pulsazione naturale,  $Q_{hp} := 0.5$  è il suo fattore di merito e  $G_{hp} := 1$  è il suo guadagno iniziale, modificabile attraverso il rispettivo potenziometro. La frequenza di taglio del filtro è  $f_{hp} := 1000$  Hz nel caso del filtro passa-alto a guadagno variabile mentre è  $f_{hp} = 1$  Hz nel caso del filtro passa-alto a coefficienti costanti.

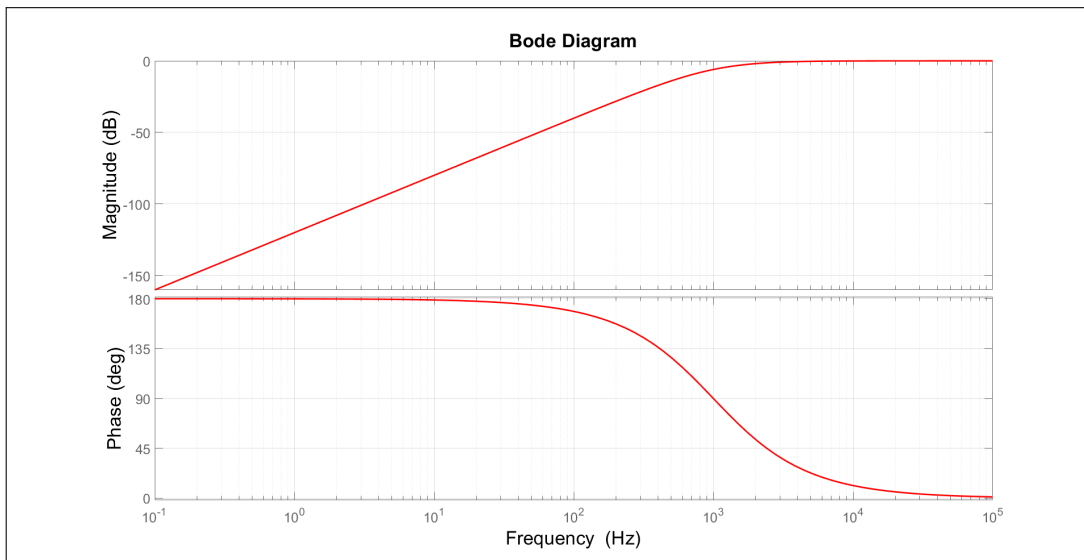


Figura 2.7: Risposta in frequenza filtro passa-alto a guadagno variabile

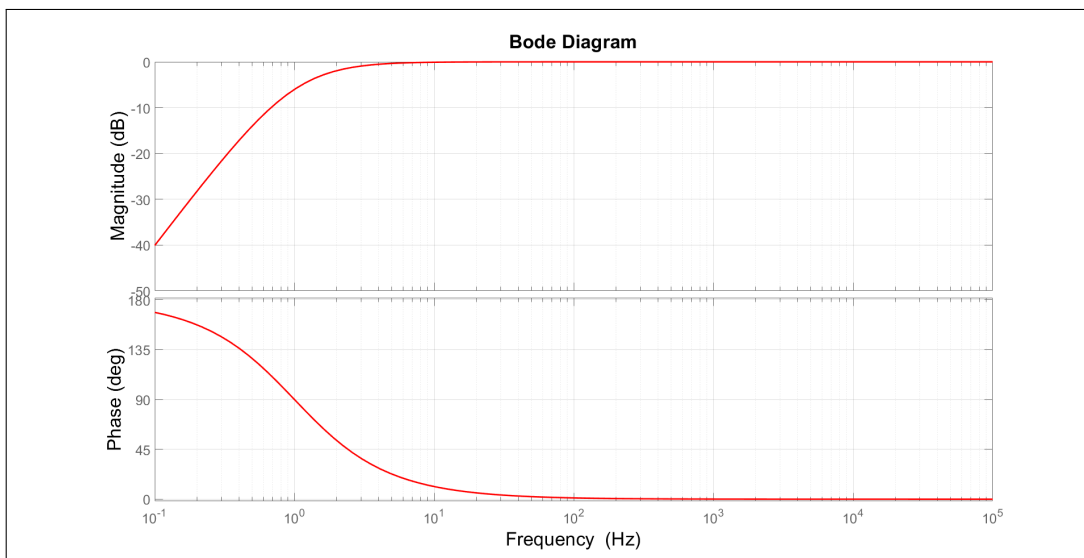


Figura 2.8: Risposta in frequenza filtro passa-alto a coefficienti costanti

Applicando la trasformata bilineare a  $W_{hp}(s)$ , si ottiene la seguente funzione di trasferimento digitale:

$$H_{hp}(z) = \frac{f_0 + f_1 z^{-1} + f_2 z^{-2}}{E_0 + E_1 z^{-1} + E_2 z^{-2}} \quad (2.9)$$

dove:  $E_0 = 4 + 2T_c \frac{\omega_{hp}}{Q_{hp}} + \omega_{hp}^2 T_c^2$ ,  $E_1 = 2\omega_{hp}^2 T_c^2 - 8$ ,  $E_2 = 4 - 2T_c \frac{\omega_{hp}}{Q_{hp}} + \omega_{hp}^2 T_c^2$ ,  $f_0 = f_2 = 4G_{hp}$ ,  $f_1 = -8G_{hp}$ .

Si noti che, nel caso dei filtri a guadagno variabile, i coefficienti del numeratore sono liberi di variare mentre i quelli del denominatore sono fissi.

La funzione di trasferimento generata dalla cascata fra il filtro passa-alto a coefficienti costanti e il parallelo dei tre filtri a guadagno variabile risulta essere come nell'equazione (2.10):

$$W(s) = W_{fhp}(s) [W_{lp}(s) + W_{bp}(s) + W_{hp}(s)] \quad (2.10)$$

La relativa funzione di trasferimento è riportata in figura (2.9):

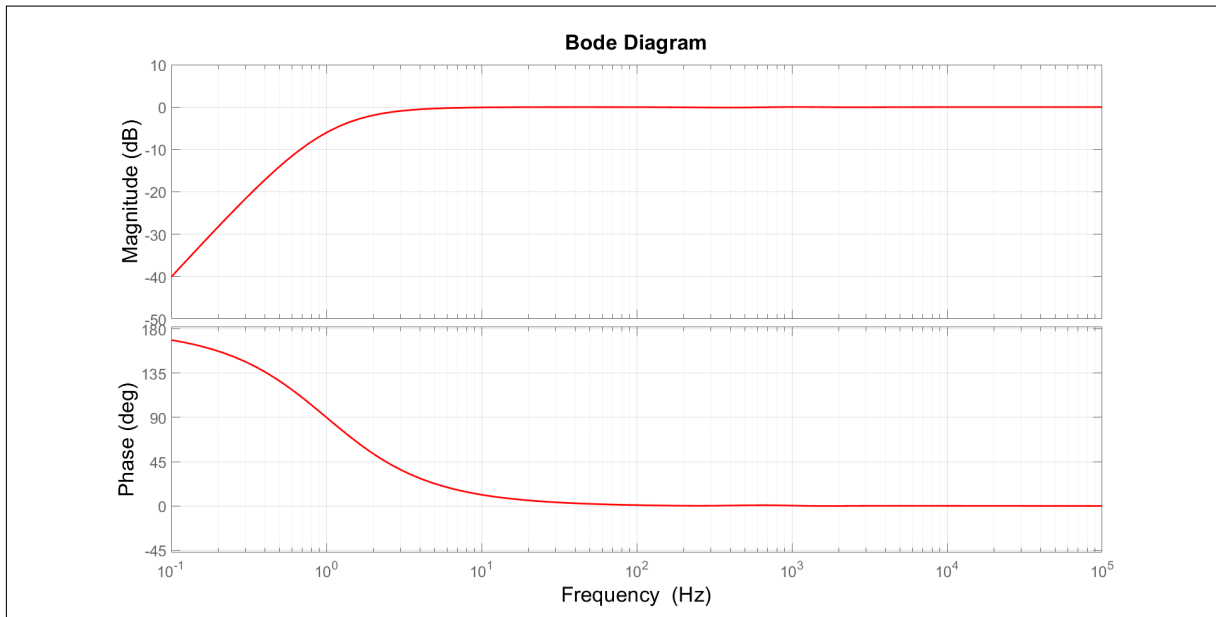


Figura 2.9: Risposta in frequenza filtro equalizzatore

Essa elimina correttamente la componente DC introdotta dal circuito di input conditioning e, in base alla posizione dei potenziometri, attenua e/o amplifica le tre bande di frequenza. La risposta in frequenza presenta un diagramma del guadagno quasi piatto a 0 dB nella banda di frequenza del segnale audio, con una variazione massima di circa -0.12dB dal valore obiettivo. Analogamente, la fase del filtro complessivo risulta quasi piatta a 0°, con una variazione massima di circa 0.5°. L'obiettivo riguardante la risposta in frequenza del filtro completo riportato nel Capitolo 1 è quindi raggiunto.

Avendo ricavato le funzioni di trasferimento dei filtri digitali, le rispettive equazioni alle differenze possono essere estratte comparando (1.1) con (2.5), (2.7) e (2.9).

- Filtri passa-basso:

$$y(n) = \frac{1}{A_0} [b_0x(n) + b_1x(n-1) + b_2x(n-2) - A_1y(n-1) - A_2y(n-2)] \quad (2.11)$$

- Filtri passa-banda:

$$y(n) = \frac{1}{C_0} [d_0x(n) + d_2x(n-2) - C_1y(n-1) - C_2y(n-2)] \quad (2.12)$$

- Filtri passa-alto a guadagno variabile:

$$y(n) = \frac{1}{E_0} [f_0x(n) + f_1x(n-1) + f_2x(n-2) - E_1y(n-1) - E_2y(n-2)] \quad (2.13)$$

- Filtri passa-alto a coefficienti costanti:

$$y(n) = \frac{1}{G_0} [h_0x(n) + h_1x(n-1) + h_2x(n-2) - G_1y(n-1) - G_2y(n-2)] \quad (2.14)$$

I coefficienti costanti possono essere calcolati per poi essere inseriti direttamente nel software del microcontrollore: i valori numerici risultanti sono riportati nella tabella (2.1).

| Nome  | Valore                                                                          |
|-------|---------------------------------------------------------------------------------|
| $A_0$ | 4.5184461916161098919441581269245223032880490229516021963582524937712027...     |
| $A_1$ | -7.9684172659165140524197296288003963163669961618968294699954772819960958565... |
| $A_2$ | 3.5131365424673760556361122442750813803449548151515683336462702242327014...     |
| $C_0$ | 4.5854726265583687341264156532650597252045007331871816487939770591022525...     |
| $C_1$ | -7.9600281021755880975937203114505015879019795174006747979630259350263088184... |
| $C_2$ | 3.4544992712660431682798640352844386868935197494121435532429970058714386...     |
| $E_0$ | 4.6529925417206820442396149041555909546777181283931231407610222919021134...     |
| $E_1$ | -7.9506519779945532069058275450006192443234315029637960468679332531188997758... |
| $E_2$ | 3.3963554802847647488545575508437898009988503686430808123710444549789868...     |
| $G_0$ | 4.0006283432047289613710890757628834002672172721641235392661715549518350...     |
| $G_1$ | -7.9999999506519779945532069058275450006192443234315029637960468679332531188... |
| $G_2$ | 3.9993717061432930440757040184095715991135384044043734969377815771149119...     |

Tabella 2.1: Valori coefficienti costanti

Viceversa, i coefficienti variabili in base al valore di guadagno verranno implementati tramite la loro definizione matematica direttamente nel software del microcontrollore.



L'attenuazione di canale verrà anch'essa implementata direttamente in software come rapporto fra il valore digitale proporzionale alla posizione del potenziometro e il valore massimo convertibile dal convertitore A/D.

Infine, l'aggiunta dell'offset DC al segnale filtrato verrà implementata tramite una semplice addizione di un termine costante.

## 2.3 Scheda Nucleo

Definite le specifiche di progetto e progettati sia i circuiti di input conditioning che i filtri digitali, si può procedere con il setup della scheda Nucleo ed alla programmazione della stessa: per questo scopo viene usata STM32CubeIDE.

Viene creato un progetto STM32 avente come scheda obiettivo la scheda NUCLEO-G431RB. Nella finestra di selezione delle periferiche da integrare con l'Human Machine Interface viene scelto di disabilitare il LED verde utente.

### 2.3.1 Configurazione microcontrollore

La prima operazione da eseguire nella finestra del Device Configuration Tool è quello di rendere disponibile la porta PA5 del microcontrollore, relativa al LED verde: essa viene impostata su Reset\_State.

Successivamente, vengono configurati i due ADC del microcontrollore. Essi vengono configurati in modo tale che ADC1 esegua conversioni su sette canali, mentre ADC2 esegua conversioni su sei canali. Considerando la configurazione degli ADC del microcontrollore in figura (1.3) e (1.4), i rispettivi canali utilizzati sono indicati in ordine di conversione nella tabella (2.2): essi sono impostati come input single-ended, ovvero la misura di tensione viene eseguita rispetto al riferimento di massa della scheda. Successivamente, nella finestra di configurazione sono definiti ulteriori parametri relativi agli ADC. Nella tabella (2.3) vengono riportati tutti i parametri impostati in comune ad entrambi ai convertitori.

| ADC1       | ADC2       |
|------------|------------|
| ADC12_INP1 | ADC2_INP3  |
| ADC12_INP2 | ADC2_INP4  |
| ADC1_INP5  | ADC2_INP5  |
| ADC12_INP7 | ADC12_INP9 |
| ADC12_INP6 | ADC12_INP8 |
| ADC1_INP11 | ADC2_INP11 |
| ADC1_INP12 |            |

Tabella 2.2: Configurazione Canali ADC

| <b>ADCs_Common_Settings</b>        |                                      |
|------------------------------------|--------------------------------------|
| Mode                               | Dual regular simultaneous mode only  |
| DMA Access Mode                    | DMA access mode enabled              |
| <b>ADC_Settings</b>                |                                      |
| Clock Prescaler                    | Synchronous clock mode divided by 4  |
| Resolution                         | ADC 12-bit resolution                |
| Data Alignment                     | Right alignment                      |
| Scan Conversion Mode               | Enabled                              |
| End Of Conversion Selection        | End of sequence of conversion        |
| DMA Continuous Requests            | Enabled                              |
| <b>ADC_Regular_ConversionMode</b>  |                                      |
| Enable Regular Conversions         | Enable                               |
| Number Of Conversion               | 7                                    |
| External Trigger Conversion Source | Timer 6 Trigger Out event            |
| External Trigger Conversion Edge   | Trigger detection on the rising edge |
| Rank                               | [1...7]                              |
| Channel                            | Channel 1,2,5,7,6,11,12 (ADC1)       |
|                                    | Channel 3,4,5,9,8,11,11 (ADC2)       |
| Sampling Time                      | 12.5 Cycles                          |

Tabella 2.3: Configurazione Parametri ADC

Si noti che la frequenza di clock degli ADC è pari a  $f_{ADC} = f_{CLK}/4 = 42.5 \text{ MHz}$  e la fine della conversione coincide con la fine della conversione della sequenza dei canali.

Sotto la voce ADC\_Regular\_ConversionMode devono anche essere definiti l'ordine di conversione dei canali e il tempo di campionamento per singola conversione. Ad ogni rango di conversione viene associato un canale ed un tempo di campionamento. Quest'ultimo sarà in comune a tutti i canali e sarà pari a 12.5 cicli di clock: in questo modo, il convertitore riuscirà a convertire la tensione in ingresso ad ogni canale con buona precisione.

Infine, viene abilitato un canale DMA per ADC1 al fine di trasferire i risultati delle conversioni direttamente in memoria. Il canale DMA è abilitato in modalità circolare e a trasferire dati di lunghezza pari ad una parola (32 bit).

In seguito, vengono abilitati i due canali relativi al convertitore digitale-analogico DAC1: essi vengono collegati esclusivamente alle relative porte del microcontrollore, ovvero PA4 e PA5.

L'ultima periferica da abilitare è il timer TIM6: esso verrà utilizzato come trigger per l'inizio della conversione degli ADC e del DAC. Nella configurazione del timer vengono definiti due parametri, entrambi numeri naturali a 16 bit: il prescaler (PSC) e il counter period, il cui valore

è salvato nell'AutoReload Register (ARR). Il primo parametro riduce la frequenza di clock del timer, dividendola per il valore inserito nel relativo registro + 1, mentre il secondo è un contatore di cicli di clock. La formula (2.15) viene usata per calcolare la frequenza di trigger del timer.

$$f_{TIM6} = \frac{f_{CLK}}{(PSC + 1)(ARR + 1)} \quad (2.15)$$

La frequenza  $f_{TIM6}$  obiettivo è pari a 40 kHz: essa è ottenuta impostando  $PSC = 169$  e  $ARR = 24$ .

Infine, viene abilitato l'update event come evento di trigger sotto la voce Trigger Output (TRGO) Parameters.

Il processo di configurazione del microcontrollore è completo: lo scheletro del codice sorgente può essere quindi generato utilizzando la funzione di generazione del codice del Device Configuration Tool.

### 2.3.2 Codice sorgente

Dopo la generazione del codice da parte del Device Configuration Tool, la cartella del progetto contiene tutti i file con codice sorgente e librerie necessarie per la corretta configurazione delle periferiche indicate in precedenza: in particolare, viene generato un file chiamato *main.c*, contenente lo scheletro del codice. In esso sono definite le funzioni e i parametri necessari per la configurazione di ADC, DAC e del timer, oltre alle altre periferiche selezionate in automatico dal Device Configuration Tool.

Data la modularità dei filtri che compongono i canali audio, viene definito uno *struct* che comprende tutti i valori di guadagno delle bande di frequenza e i valori di ingresso ed uscita di ogni filtro digitale.

```
/* USER CODE BEGIN PTD */
typedef struct AudioCh {
    uint16_t    xL[3], xR[3];
    float       G_lp, G_bp, G_hp, G,
               yL[3], yR[3],
               zL_LP[3], zR_LP[3],
               zL_BP[3], zR_BP[3],
               zL_HP[3], zR_HP[3],
               zL, zR;
} AudioCh;
/* USER CODE END PTD */
```

Listato 2.1: Struct canali audio

Vengono poi inserite le definizioni del numero di conversioni per ogni ADC e i valori dei coefficienti costanti dei filtri digitali in riferimento alla tabella (2.1), ad esclusione dei coefficienti che compaiono come denominatori di frazione. Nel loro caso, si opta direttamente per l'implementazione del valore inverso dei coefficienti: in questo modo, si solleva la FPU dal calcolo di divisioni fra numeri a virgola mobile trasformandole in moltiplicazioni, le quali sono più leggere da un punto di vista computazionale.

```

/* USER CODE BEGIN PD */
#define ADC_CONVERSION_NUMBER 7

// Coefficienti filtro passa basso
#define A0Inv 0.22371316367261522121f
#define A1 -7.97389856687315210943f
#define A2 3.55609178513218129241f
#define B0 0.01305071656342394528f
#define B1 0.02610143312684789056f
#define B2 0.01305071656342394528f

// Coefficienti filtro passa banda
#define C0Inv 0.22072150672837394881f
#define C1 -7.9669653736988331385f
#define C2 3.50243851529043544627f
#define D0 0.56548667764616278292f
#define D2 -0.56548667764616278292f

// Coefficienti filtro passa alto
#define E0Inv 0.21778946093655217558f
#define E1 -7.95921651073930017099f
#define E2 3.44919308034129659841f
#define F0 4.0f
#define F1 -8.0f
#define F2 4.0f

// Coefficienti filtro passa alto fisso
#define G0Inv 0.24996430390657009305f
#define G1 -7.9999999592165107393f
#define G2 3.99942882172745557703f
#define H0 4.0f
#define H1 -8.0f
#define H2 4.0f
/* USER CODE END PD */

```

Listato 2.2: Definizione valori numerici coefficienti filtri digitali

L'utilizzo del suffisso  $f$  dopo ogni valore numerico è imposto dal suo formato obiettivo. In questo caso, i coefficienti devono essere implementati come numeri a virgola mobile a precisione singola (float), in modo tale da poter essere utilizzati dalla FPU del microcontrollore come operandi validi. L'implementazione dei coefficienti come numeri a virgola mobile implica, però, una riduzione della loro precisione rispetto ai valori ideali, data la parziale troncatura della parte decimale causata dal limite di memoria imposto dal microcontrollore.

In seguito vengono definite le variabili globali del progetto. Esse sono:

- due variabili intere senza segno a 16 bit ciascuna, usate per salvare il valore da trasmettere al DAC alla fine dell'operazione di filtraggio;
- un array di numeri interi senza segno a 32 bit, usato come buffer per ricevere i dati provenienti dagli ADC al termine della loro conversione;
- una variabile float, usata per salvare il valore di attenuazione generale dei segnali filtrati;
- due struct AudioCh, usati per generare tutte le variabili necessarie alla definizione di due canali stereo.

```
/* USER CODE BEGIN PV */
uint16_t      outL, outR;
uint32_t      adcDMABuffer[ADC_CONVERSION_NUMBER];
float         masterGain;
AudioCh       ch1, ch2;
/* USER CODE END PV */
```

Listato 2.3: Variabili globali

Vengono poi definiti i prototipi delle funzioni ausiliarie.

```
/* USER CODE BEGIN PFP */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc);
void AudioCh_DataUpdate(AudioCh* ch);
float AudioCh_SecondOrderLP(AudioCh* ch, float input[3], float output[3]);
float AudioCh_SecondOrderBP(AudioCh* ch, float input[3], float output[3]);
float AudioCh_SecondOrderHP(AudioCh* ch, float input[3], float output[3]);
float AudioCh_FixedSecondOrderHP(uint16_t input[3], float output[3]);
/* USER CODE END PFP */
```

Listato 2.4: Prototipi funzioni ausiliarie

Viene quindi implementata la funzione *main* del programma. Nella sezione User Code 2 vengono eseguite le seguenti operazioni:

- auto-calibrazione di ADC1 ed ADC2;

- attivazione di ADC1 ed ADC2 in modalità duale con trasferimento tramite DMA dei risultati delle conversioni;
- attivazione del DAC1 in modalità duale;
- attivazione del timer.

```

/* USER CODE BEGIN 2 */
// Auto-calibrazione ADC
HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);
HAL_ADCEx_Calibration_Start(&hadc2, ADC_SINGLE_ENDED);

// Attivazione ADC in modalità duale
HAL_ADCEx_MultiModeStart_DMA(&hadc1, adcDMABuffer, ADC_CONVERSION_NUMBER);

// Attivazione DAC in modalità duale
HAL_DACEx_DualStart(&hdac1);

// Avvio timer
HAL_TIM_Base_Start(&htim6);
/* USER CODE END 2 */

```

Listato 2.5: Sezione User Code 2

Vengono quindi implementate le funzioni ausiliarie, ovvero le funzioni di callback al momento del completamento della conversione da parte degli ADC, le funzioni usate per aggiornare e calcolare i parametri dei filtri digitali e le funzioni per il calcolo dei valori di uscita di ogni filtro digitale. Esse sono definite nella sezione User Code 4 del codice sorgente.

La funzione di callback per la conversione completa degli ADC si occupa di eseguire il filtraggio dei segnali e di aggiornare il valore in uscita dal sistema da far convertire al DAC.

```

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    // Aggiornamento dati filtri, estrazione e calcolo nuovi campioni
    AudioCh_DataUpdate(&ch1);
    AudioCh_DataUpdate(&ch2);

    // Calcolo guadagno master
    masterGain = ((uint16_t)adcDMABuffer[6]) * 0.0002442002442002442f;
    // Somma segnali filtrati e aggiunta offset DC per DAC
    outL = ((&ch1)->zL + (&ch2)->zL) * masterGain + 2048;
    outR = ((&ch1)->zR + (&ch2)->zR) * masterGain + 2048;

    // Check limiti valori in uscita dal DAC
    if (outL > 4095) outL = 4095;
}

```

```

else if (outL < 0) outL = 0;
else if (outR > 4095) outR = 4095;
else if (outR < 0) outR = 0;

// Set nuovi campioni uscita DAC
HAL_DACEx_DualSetValue(&hdac1, DAC_ALIGN_12B_R, outL, outR);
}

```

Listato 2.6: Funzione callback conversione ADC

Viene implementata una funzione ausiliaria chiamata *AudioCh\_DataUpdate*, la quale si occupa di aggiornare i dati di ingressi ed uscite di ogni filtro, estrarre i campioni audio nuovi dal buffer DMA, calcolare i valori di guadagno delle tre bande di frequenza dei filtri e eseguire il filtraggio dei segnali audio. Inizialmente, la funzione definisce il canale indicato come argomento della funzione: in questo modo, i valori estratti dal buffer DMA sono coerenti con il canale selezionato. In base all'argomento della funzione, una variabile d'appoggio assume il valore 0 o 1: essa viene usata per impostare un offset sull'indice delle locazioni del buffer DMA in modo da selezionare i dati corretti. La configurazione del buffer DMA viene indicata in tabella (2.4).

| ADC2  | ADC1       |
|-------|------------|
| Ch1L  | Ch1R       |
| Ch2L  | Ch2R       |
| Ch1HP | Ch1BP      |
| Ch2HP | Ch2BP      |
| Ch1LP | Ch1G       |
| Ch2LP | Ch2G       |
| Ch2LP | masterGain |

Tabella 2.4: Configurazione Buffer DMA

Il canale ADC che campiona il potenziometro relativo al filtro passa-basso del secondo canale viene campionato due volte di fila: questo per ottenere un numero uguale di conversioni fra i due canali, condizione necessaria alla corretta esecuzione delle conversioni ADC in modalità duale.

Per estrarre i valori convertiti dai singoli ADC è necessario dividere ogni locazione del buffer in due celle da 16 bit: in questo modo, i primi 16 bit indicano un dato convertito dal ADC1, mentre i 16 bit rimanenti indicano un dato convertito dal ADC2. Nel primo caso è sufficiente un cast implicito da numero intero senza segno a 32 bit a

numero intero senza segno a 16 bit mentre, nel secondo caso, è necessario eseguire uno shift di 16 bit verso destra del valore a 32 bit.

Per calcolare il guadagno di ogni banda di frequenza viene definita una funzione *gain* :  $[0, 4095] \rightarrow [0.5, 2]$  in modo che, quando la tensione proveniente da un potenziometro è minima, il guadagno è pari a 0.5 mentre, quando la tensione proveniente da un potenziometro è

massima, il guadagno è pari a 2. Essa viene definita come nell'equazione (2.16).

$$gain(n) = \begin{cases} 0.5 + \frac{n}{4094} & , n \in [0, 2047] \\ \frac{n-1}{2047} & , n \in [2048, 4095] \end{cases} \quad (2.16)$$

Anche in questo caso, le divisioni vengono implementate tramite moltiplicazioni usando come operando l'inverso del divisore.

Il calcolo del guadagno di canale viene calcolato come il rapporto fra il valore intero convertito e il valore massimo per un numero a 12 bit, ovvero 4095.

Viene poi eseguito il filtraggio di tutti i segnali coinvolti nel canale utilizzando ulteriori funzioni ausiliarie definite in seguito.

```
void AudioCh_DataUpdate(AudioCh* ch) {
    // Aggiornamento campioni I/O filtri lato sinistro
    ch->xL[2] = ch->xL[1];
    ch->xL[1] = ch->xL[0];
    ch->yL[2] = ch->yL[1];
    ch->yL[1] = ch->yL[0];
    ch->zL_LP[2] = ch->zL_LP[1];
    ch->zL_LP[1] = ch->zL_LP[0];
    ch->zL_BP[2] = ch->zL_BP[1];
    ch->zL_BP[1] = ch->zL_BP[0];
    ch->zL_HP[2] = ch->zL_HP[1];
    ch->zL_HP[1] = ch->zL_HP[0];
    // Aggiornamento campioni I/O filtri lato destro
    ch->xR[2] = ch->xR[1];
    ch->xR[1] = ch->xR[0];
    ch->yR[2] = ch->yR[1];
    ch->yR[1] = ch->yR[0];
    ch->zR_LP[2] = ch->zR_LP[1];
    ch->zR_LP[1] = ch->zR_LP[0];
    ch->zR_BP[2] = ch->zR_BP[1];
    ch->zR_BP[1] = ch->zR_BP[0];
    ch->zR_HP[2] = ch->zR_HP[1];
    ch->zR_HP[1] = ch->zR_HP[0];

    // Calcolo guadagni ed estrazione nuovi valori campionati
    int i = 0;
    if (ch == &ch1) i = 0;
    else if (ch == &ch2) i = 1;
    ch->xL[0] = adcDMABuffer[i] >> 16;
    ch->xR[0] = (uint16_t)adcDMABuffer[i];
}
```



```

ch->G_hp = ((adcDMABuffer[i+2] >> 16) < 2048) ? (0.5f + 0.00024425989252564728f * (
    adcDMABuffer[i+2] >> 16)) : (0.0004885197850512945f * ((adcDMABuffer[i+2] >>
    16) - 1));
ch->G_bp = ((uint16_t)adcDMABuffer[i+2] < 2048) ? (0.5f + 0.00024425989252564728f *
    (uint16_t)adcDMABuffer[i+2]) : 0.0004885197850512945f * ((uint16_t)
    adcDMABuffer[i+2] - 1);
ch->G_lp = ((adcDMABuffer[i+4] >> 16) < 2048) ? (0.5f + 0.00024425989252564728f * (
    adcDMABuffer[i+4] >> 16)) : 0.0004885197850512945f * ((adcDMABuffer[i+4] >> 16)
    - 1);
ch->G = ((uint16_t)adcDMABuffer[i+4]) * 0.0002442002442002442f;

// Filtraggio segnali digitali
ch->yL[0] = AudioCh_FixedSecondOrderHP(ch->xL, ch->yL);
ch->yR[0] = AudioCh_FixedSecondOrderHP(ch->xR, ch->yR);
ch->zL_LP[0] = AudioCh_SecondOrderLP(ch, ch->yL, ch->zL_LP);
ch->zL_BP[0] = AudioCh_SecondOrderBP(ch, ch->yL, ch->zL_BP);
ch->zL_HP[0] = AudioCh_SecondOrderHP(ch, ch->yL, ch->zL_HP);
ch->zL = ch->G * (ch->zL_LP[0] + ch->zL_BP[0] + ch->zL_HP[0]);
ch->zR_LP[0] = AudioCh_SecondOrderLP(ch, ch->yR, ch->zR_LP);
ch->zR_BP[0] = AudioCh_SecondOrderBP(ch, ch->yR, ch->zR_BP);
ch->zR_HP[0] = AudioCh_SecondOrderHP(ch, ch->yR, ch->zR_HP);
ch->zR = ch->G * (ch->zR_LP[0] + ch->zR_BP[0] + ch->zR_HP[0]);
}

```

Listato 2.7: Funzione AudioCh\_DataUpdate

Infine, vengono riportate le funzioni ausiliarie che implementano i singoli filtri digitali.

```

float AudioCh_SecondOrderLP(AudioCh* ch, float input[3], float output[3]) {
    return A0Inv * ((ch->G_lp)*B0*input[0] + (ch->G_lp)*B1*input[1] + (ch->G_lp)*B2*
        input[2] - A1*output[1] - A2*output[2]);
}

```

Listato 2.8: Filtro passa-basso digitale

```

float AudioCh_SecondOrderBP(AudioCh* ch, float input[3], float output[3]) {
    return C0Inv * ((ch->G_bp)*D0*input[0] + (ch->G_bp)*D2*input[2] - C1*output[1] - C2
        *output[2]);
}

```

Listato 2.9: Filtro passa-banda digitale

```
float AudioCh_SecondOrderHP(AudioCh* ch, float input[3], float output[3]) {
    return EOInv * ((ch->G_hp)*F0*input[0] + (ch->G_hp)*F1*input[1] + (ch->G_hp)*F2*
        input[2] - E1*output[1] - E2*output[2]);
}
```

Listato 2.10: Filtro passa-alto digitale

```
float AudioCh_FixedSecondOrderHP(uint16_t input[3], float output[3]) {
    return GOInv * (H0*input[0] + H1*input[1] + H2*input[2] - G1*output[1] - G2*output
        [2]);
}
```

Listato 2.11: Filtro passa-alto digitale a coefficienti fissi

Il file *main.c* completo è riportato in Appendice A.

Prima di programmare la scheda sono eseguite altre due operazioni:

- viene attivato il livello di ottimizzazione -O1 del compilatore: in questo modo, il codice viene leggermente ottimizzato per ridurre il tempo di esecuzione del programma;
- viene rimosso un Solder Bridge (SB) dalla Scheda Nucleo. Esso è una resistenza  $0\ \Omega$  SMD che agisce come connettore fra parti della scheda o selettore di impostazioni lato hardware. Il SB da rimuovere è SB6, il quale viene usato per collegare la porta PA5 del microcontrollore al LED verde integrato sulla scheda. La rimozione del SB isola le due parti precedentemente connesse e consente di usare liberamente la porta PA5.

La scheda Nucleo può essere quindi programmata collegandola al PC tramite un cavo USB.

# Capitolo 3

## Collaudo

Nel capitolo che segue verranno collaudate le componenti progettate. Sono state sostenute alcune sessioni di laboratorio atte a verificare il corretto funzionamento delle parti del progetto. Verranno mostrati grafici ed immagini che illustrano il comportamento reale dei sistemi implementati.

In queste sessioni di laboratorio è stata usata la seguente strumentazione:

- oscilloscopio digitale;
- alimentatore DC da banco;
- generatore di forme d'onda arbitraria.

### 3.1 Input conditioning

L'obiettivo del collaudo del circuito di input conditioning è quello di verificarne il punto di operazione DC e la risposta in frequenza.

Inizialmente, il circuito di input conditioning è stato montato su una breadboard: viene utilizzato solo un amplificatore operazionale dei due a disposizione sul circuito integrato MCP6002.

In seguito, la tensione di alimentazione del circuito e l'uscita del generatore di forme d'onda sono state collegate ai rispettivi componenti.

Per misurare il punto operativo DC del circuito, non si applica alcun segnale arbitrario al suo ingresso. La tensione misurata usando l'oscilloscopio corrisponde quindi al valore di offset DC generato dal circuito. Essa risulta essere di circa 1.62 V: la differenza dal valore teorico implementabile è di circa 40 mV. Lo scostamento del valore di offset da quello ideale risulta essere tollerabile per questa applicazione, essendo la differenza fra i due valori ridotta.

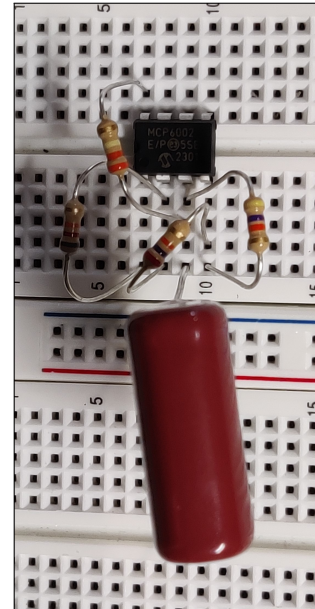


Figura 3.1: Circuito input conditioning su breadboard

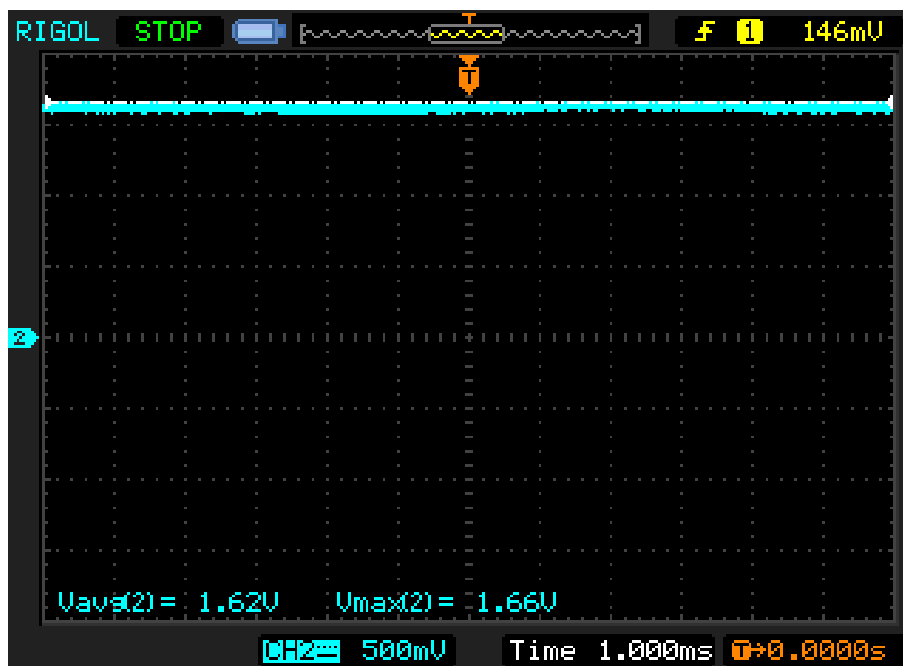


Figura 3.2: Punto operativo DC circuito input conditioning

Successivamente, è stata tracciata la risposta in frequenza. Viene applicata in ingresso al circuito un'onda sinusoidale di ampiezza picco-picco pari a 200 mV, ne viene variata la frequenza e viene osservata con l'oscilloscopio la tensione in uscita. In figura (3.3) viene riportato un esempio di confronto tensione ingresso-uscita nel caso in cui venga applicata un'onda sinusoidale con frequenza pari a 50 Hz.

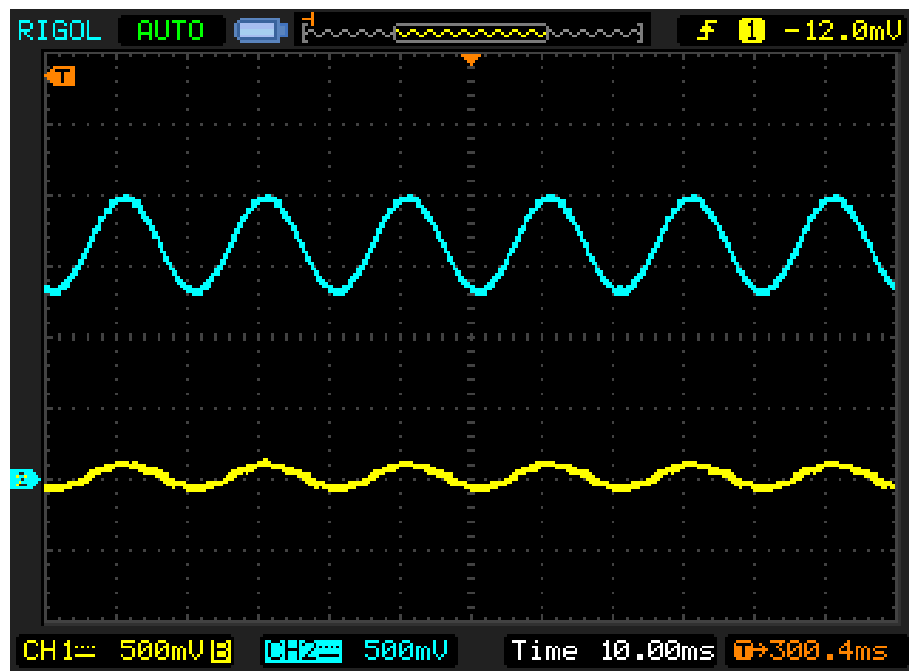


Figura 3.3: Esempio di funzionamento circuito input conditioning (50 Hz)

La risposta in frequenza viene tracciata considerando l'intervallo di frequenze da 0.1 Hz a 30kHz: viene posta particolare attenzione al suo andamento nelle vicinanze della frequenza di taglio, considerando dieci valori nella decade precedente e successiva alla frequenza di taglio. In figura (3.4) viene riportato il confronto fra la risposta in frequenza ideale e quella reale.

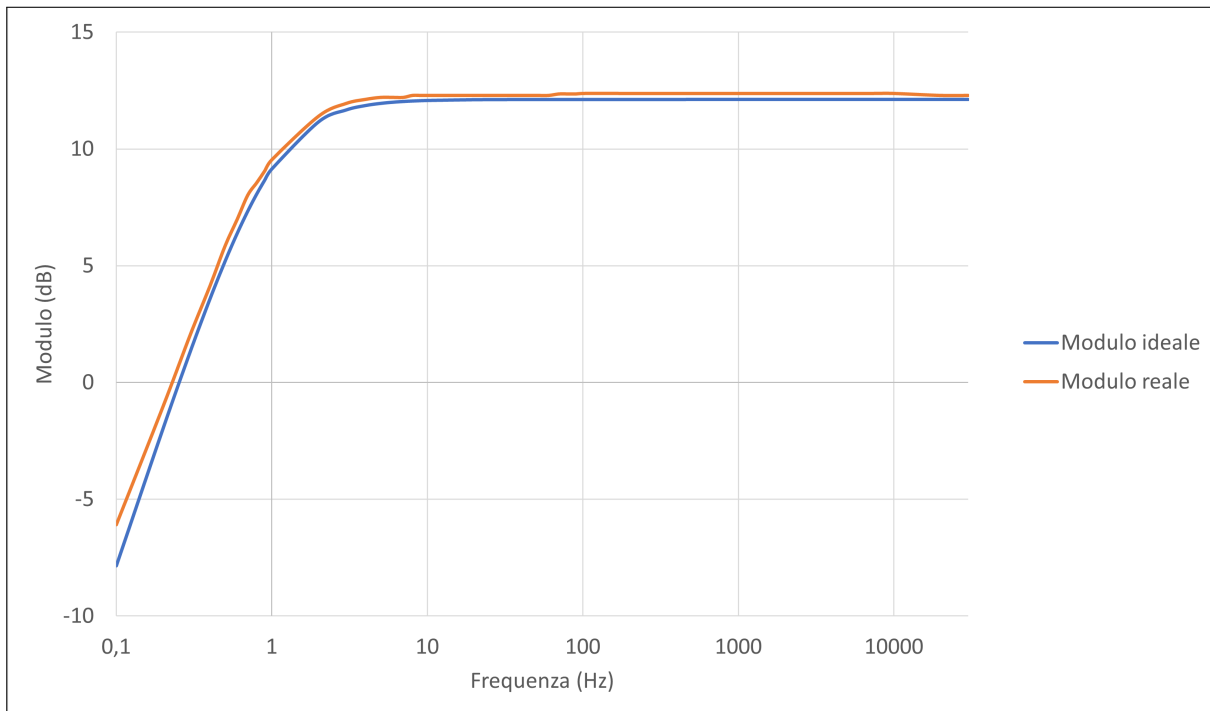


Figura 3.4: Confronto risposta in frequenza input conditioning

La risposta in frequenza reale approssima adeguatamente quella ideale: il collaudo del circuito di input conditioning viene ritenuto soddisfacente.

## 3.2 Scheda Nucleo

Gli obiettivi del collaudo della scheda Nucleo sono la verifica del corretto funzionamento delle sue periferiche, il tracciamento delle risposte in frequenza di tutti i filtri implementati e la verifica del loro funzionamento.

Come prima verifica di funzionamento delle periferiche della scheda Nucleo, viene usato il codice sorgente completo per filtrare un segnale sinusoidale con frequenza pari a 10 kHz: esso è utile a misurare con un buon grado di precisione la frequenza di campionamento del sistema.

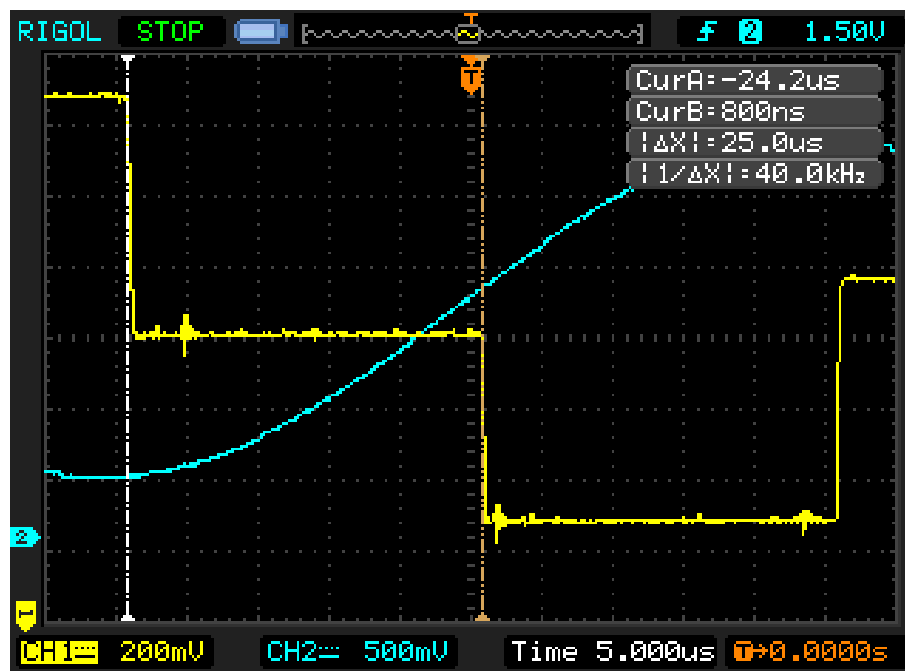


Figura 3.5: Misurazione frequenza di campionamento

Come si può vedere in figura (3.5), la distanza temporale fra due campioni di segnale convertiti dal DAC è pari a  $25 \mu\text{s}$ , ovvero la loro frequenza di aggiornamento è pari a 40 kHz. Dato che il DAC e gli ADC condividono lo stesso segnale di trigger, questo significa che il timer genera correttamente un Update Event come da specifiche di progetto.

Vengono poi tracciati i moduli delle risposte in frequenza dei singoli filtri digitali: viene creato un codice ad-hoc a questo scopo. Vengono usati un canale ADC per campionare il segnale in ingresso e un canale DAC per misurare il risultato dell'operazione di filtraggio attraverso l'uso di un oscilloscopio: la frequenza di campionamento, il tempo di sampling e le impostazioni riguardanti il DAC rimangono invariate rispetto a quelle definite nel Capitolo 2. In ingresso all'ADC viene posto un segnale sinusoidale di ampiezza picco-picco tale per cui esso occupi al meglio l'intervallo di tensione campionabile dal convertitore e per cui il segnale filtrato rispetti

i limiti di tensione massima del DAC: inoltre, al segnale sinusoidale viene sommato un offset DC pari a 1,65 V.

Di seguito sono riportati diagrammi di Bode dei filtri digitali nell'intervallo di frequenza da 0.1 Hz a 30 kHz. La risposta in frequenza di alcuni filtri è troncata ai rispettivi estremi: questo a causa dell'eccessiva attenuazione introdotta dal filtro e/o dalla vicinanza della frequenza del segnale usato in ingresso rispetto al limite imposto dalla frequenza di campionamento.

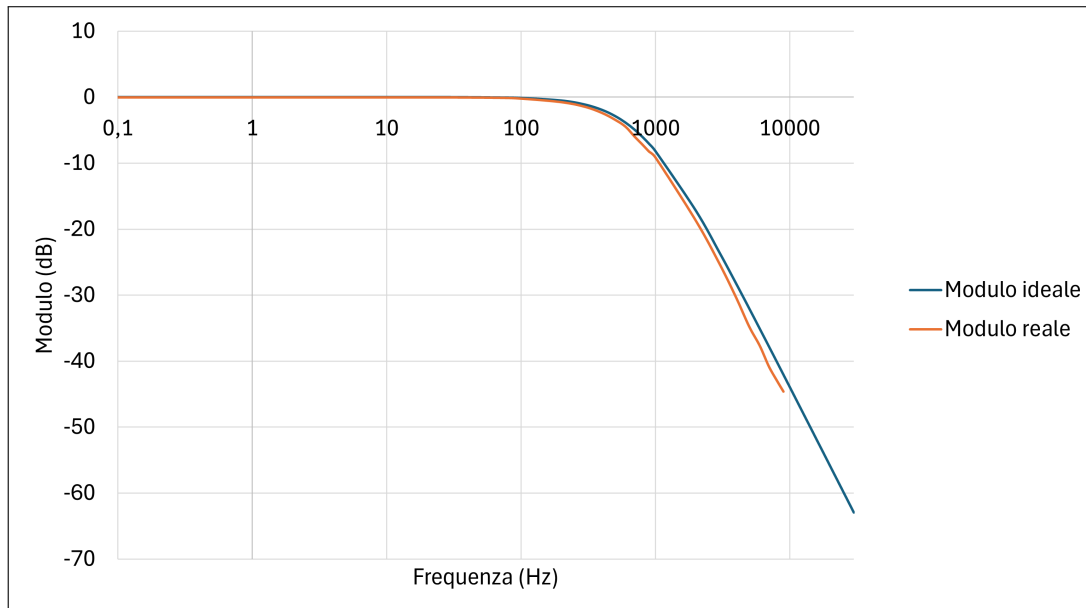


Figura 3.6: Confronto risposta in frequenza filtro passa basso digitale

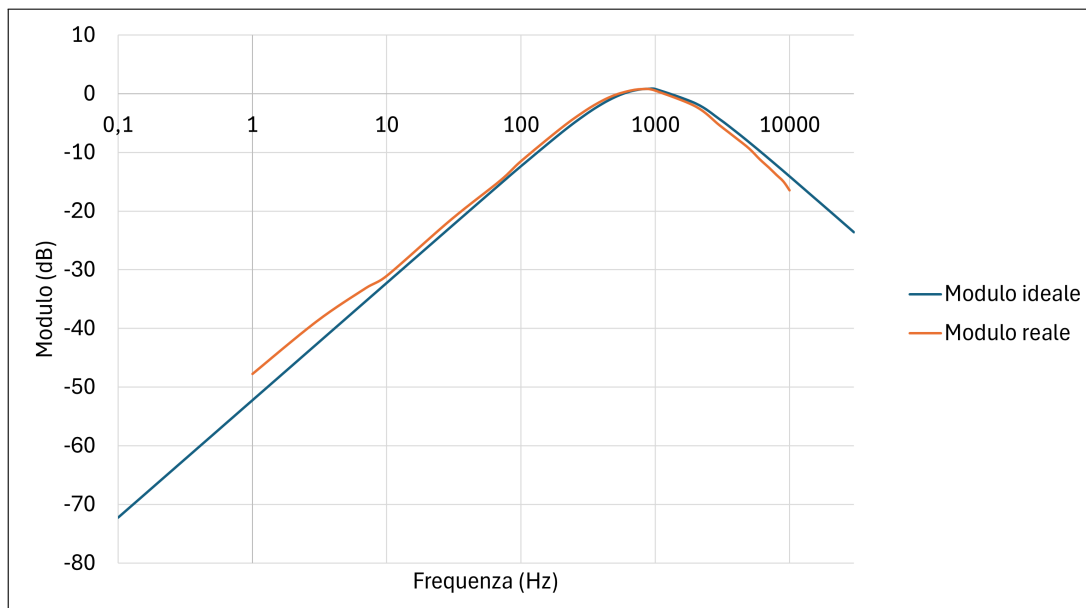


Figura 3.7: Confronto risposta in frequenza filtro passa banda digitale



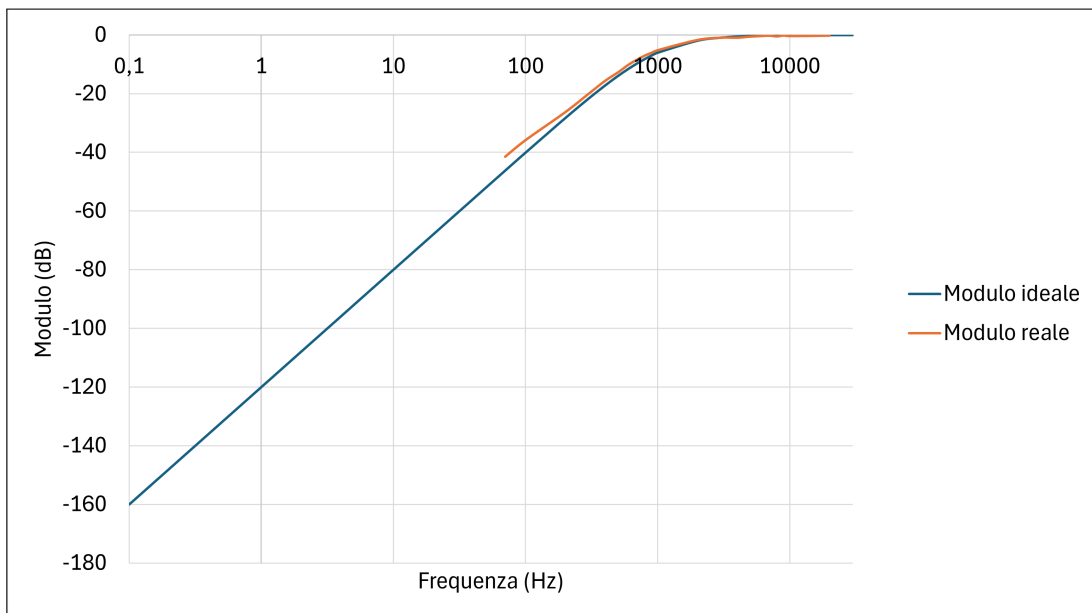


Figura 3.8: Confronto risposta in frequenza filtro passa alto digitale

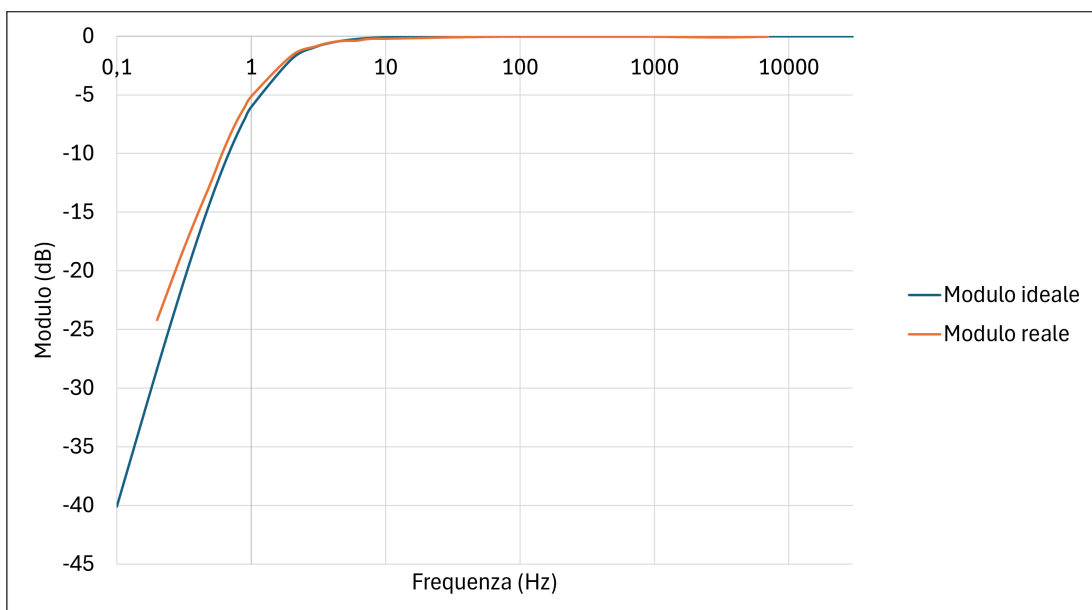


Figura 3.9: Confronto risposta in frequenza filtro passa alto fisso digitale

I filtri digitali approssimano adeguatamente le funzioni di trasferimento ideali: come indicato nel Capitolo 1, la trasformata bilineare introduce una deformazione non lineare della risposta in frequenza, ma essa risulta essere trascurabile nell'intervallo di frequenze interessato dai filtri.

In seguito, viene estratto il modulo della risposta in frequenza del filtro equalizzatore completo: viene usato il codice sorgente abilitando solo un canale alla conversione. Viene anche assemblato il circuito, compreso di circuiti di input conditioning, potenziometri e scheda Nucleo.

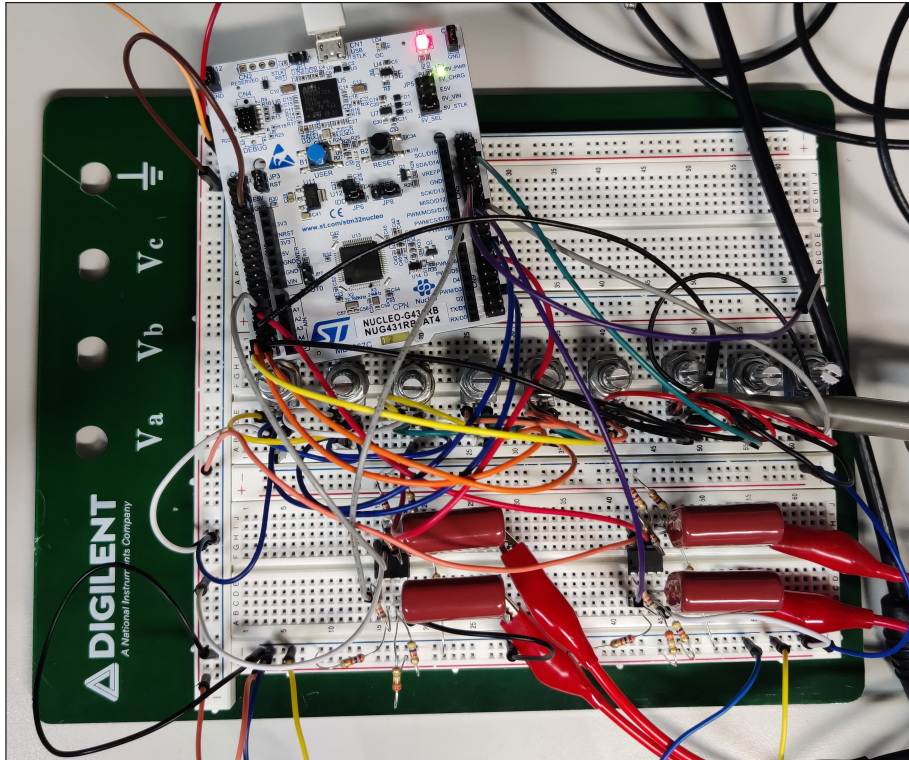


Figura 3.10: Circuito completo su breadboard

In particolare, tutti i potenziometri sono posizionati in modo tale che tutti i valori di guadagno del sistema, sia dei filtri che dell'ampiezza dei segnali filtrati, siano circa pari ad uno.

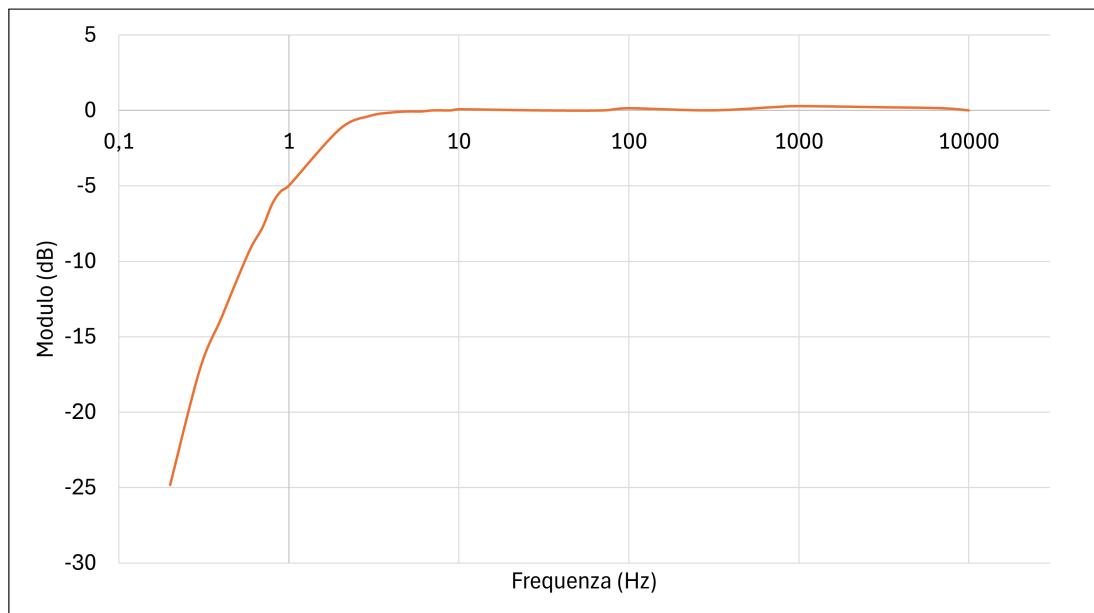


Figura 3.11: Risposta in frequenza filtro equalizzatore

Anch'esso approssima adeguatamente la risposta in frequenza ideale del filtro: essa presenta, come previsto nella fase di implementazione, leggere variazioni di ampiezza rispetto all'asse del guadagno unitario in banda audio.

Successivamente, sono state tracciate delle risposte in frequenza in casi in cui il valore di guadagno di alcune bande di frequenza non fosse unitario.

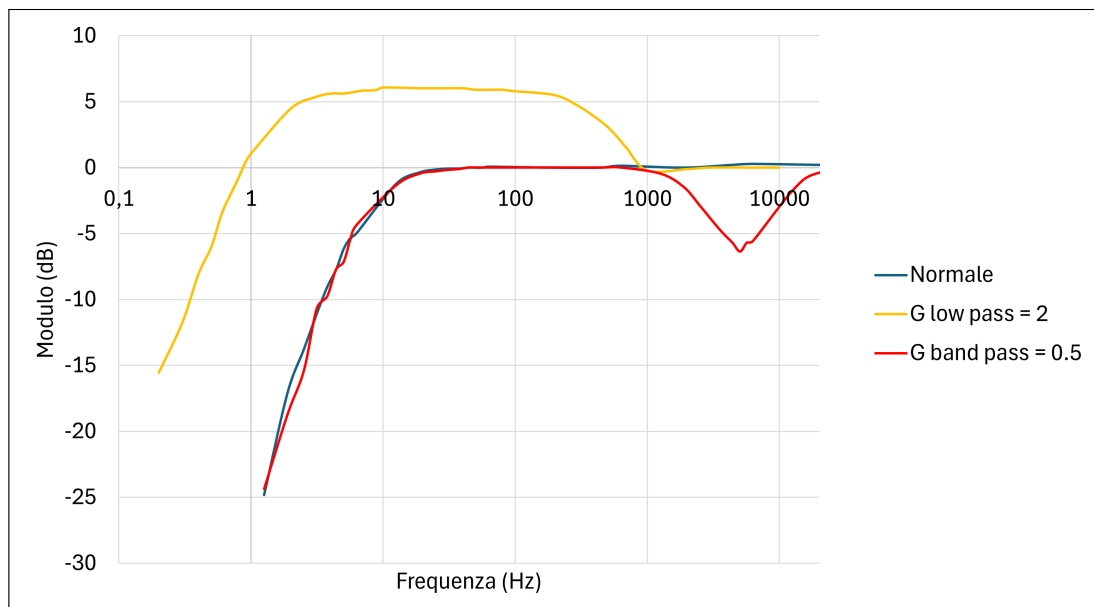


Figura 3.12: Confronto risposte in frequenza filtro equalizzatore

Sono stati estratti anche due diagrammi FFT al fine di dimostrare la corretta operazione di filtraggio del sistema. A questo fine, tutti i potenziometri sono stati posizionati come nel caso della estrazione della risposta in frequenza del filtro equalizzatore e viene usata la seguente configurazione di segnali in ingresso al mixer.

| Canale audio   | Frequenza del segnale in ingresso |
|----------------|-----------------------------------|
| Ch1 - Sinistra | 100 Hz                            |
| Ch1 - Destra   | 200 Hz                            |
| Ch2 - Sinistra | 300 Hz                            |
| Ch2 - Destra   | 400 Hz                            |

Tabella 3.1: Segnali base per grafici FFT

Tutti i segnali sono sinusoidi pure con ampiezza picco-picco pari a 400 mV.

Vista la configurazione dei segnali, il risultato dell'operazione di filtraggio dovrebbe presentare, sul lato sinistro, una sinusoide con frequenza pari a 100 Hz sovrapposta ad un'altra sinusoide con frequenza pari a 300 Hz; mentre, sul lato destro, l'uscita dovrebbe presentare la sovrapposizione di due sinusoidi con rispettive frequenze pari a 200 Hz e 400 Hz.

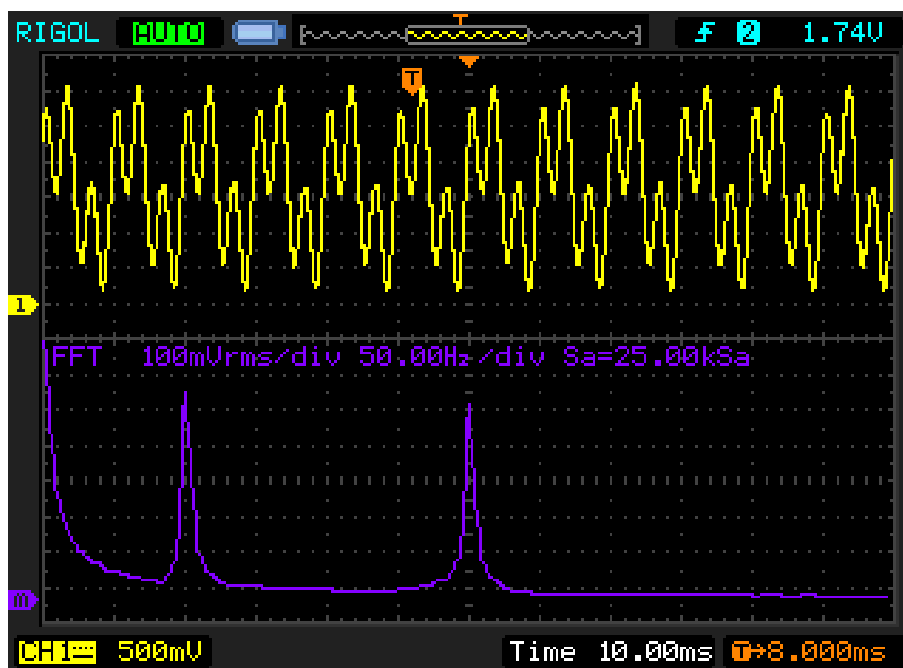


Figura 3.13: FFT canale sinistro

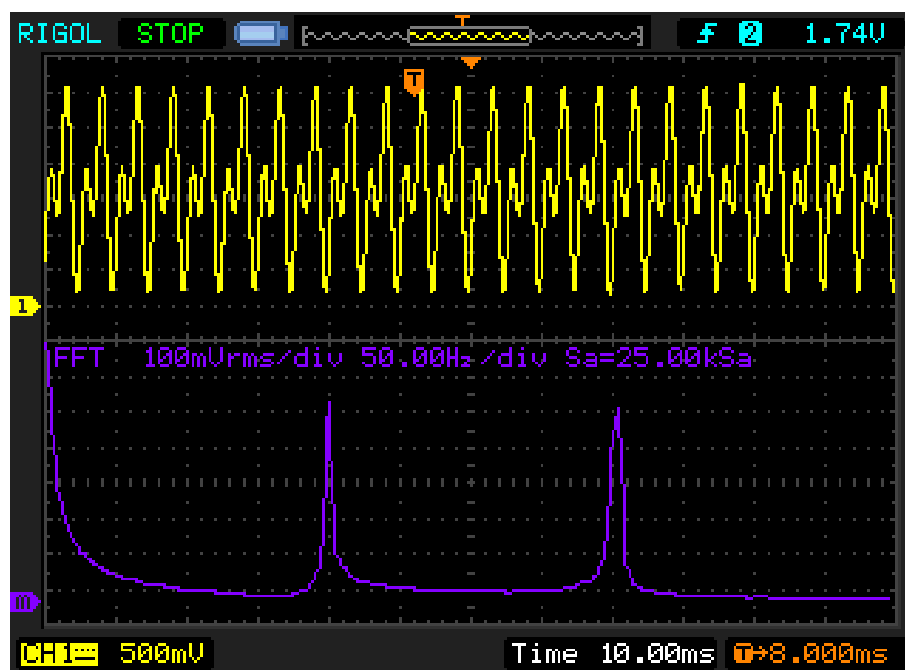


Figura 3.14: FFT canale destro

I grafici confermano il corretto filtraggio dei segnali.

Alcune imperfezioni nell'operazione di filtraggio risultano notevoli all'avvicinarsi della frequenza del segnale in ingresso al limite imposto dalla frequenza di campionamento. Ad esempio, se esso ha frequenza pari a 20 kHz, un risultato del filtraggio è riportato in figura (3.15).

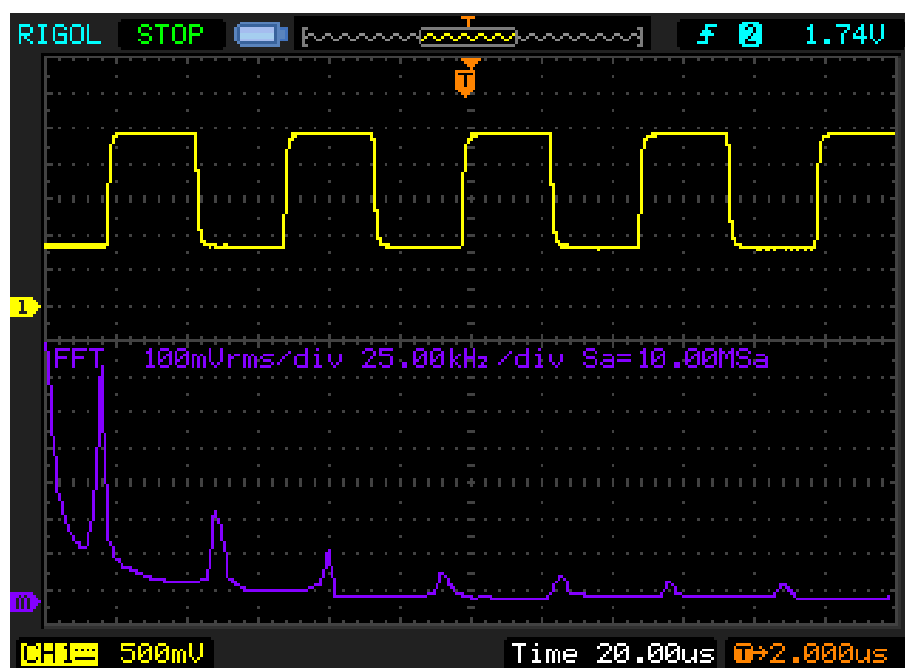


Figura 3.15: FFT con segnale a 20 kHz in ingresso

Questo fenomeno è direttamente collegabile alla vicinanza della frequenza del segnale in ingresso al limite di frequenza correttamente campionabile dal sistema. Il fatto che esso riesca a campionare la sinusoide non implica però che il segnale ricostruito dal DAC presenti una forma d'onda simile a quella di partenza: infatti, il segnale risultante è molto simile ad un'onda quadra con frequenza pari a 20 kHz. Lo spettro di frequenza occupato dal segnale presenta, oltre all'armonica principale a 20 kHz, componenti sinusoidali additive. Questo implica che, all'avvicinarsi della frequenza di un segnale in ingresso alla frequenza limite, il segnale filtrato risulterà essere distorto.

Le misurazioni sul sistema risultano essere coerenti con le limitazioni imposte in fase di progettazione.

# Capitolo 4

## Conclusioni

Il progetto risulta essere correttamente funzionante: le periferiche della scheda Nucleo funzionano a dovere, le risposte in frequenza estratte approssimano adeguatamente il loro andamento ideale e l'interfaccia fisica consente di modificare con successo il comportamento dei filtri. Nei limiti imposti dalla frequenza di campionamento, il sistema esegue con successo la sua funzione.

Di seguito, sono elencati alcuni tra i miglioramenti possibili per questo progetto.

- Aumento della frequenza di campionamento del segnale audio e della risoluzione degli ADC e dei DAC: in questo modo, la qualità del segnale campionato, filtrato e poi ricostruito risulterebbe essere di qualità maggiore. Ad esempio, per un file audio scritto su un CD, la frequenza di campionamento si aggira sui 44 kHz e ogni campione ha 16 bit di risoluzione[13]. Questo incremento deve tenere in considerazione i limiti computazionali della piattaforma digitale utilizzata.
- Aumento della quantità di canali audio: vista la loro modularità, l'aggiunta di altri canali audio risulterebbe facilitata. Questo aumento deve tenere in considerazione le necessità sulle periferiche e sulla potenza computazionale del microcontrollore utilizzato, oltre che ai circuiti fisici.
- Utilizzo di filtri FIR per la creazione del filtro equalizzatore. I filtri FIR (Finite Impulse Response) possono essere progettati in modo tale da partizionare in maniera più netta le bande di frequenza rispetto ai filtri IIR usati in questo progetto: questo comporta però una necessità computazionale maggiore.
- Creazione di un circuito stampato dedicato: esso consentirebbe di avere a disposizione un'interfaccia fissa e sicura con il mixer rispetto alla versione assemblata su una bread-board.

- Sviluppo o integrazione di un circuito di amplificazione di potenza: un modo per avere un riscontro sonoro del processo di filtraggio dei segnali audio è quello di disporre di altoparlanti, i quali però non possono essere pilotati direttamente dal microcontrollore a causa della scarsa potenza disponibile in uscita dal DAC. Questo obiettivo può essere raggiunto integrando al sistema un amplificatore di potenza, il quale si occupa di fornire la potenza necessaria agli altoparlanti per funzionare, riducendo il più possibile la distorsione del segnale filtrato.



# Appendice A

## Codice sorgente

```
1  /* USER CODE BEGIN Header */
2  /**
3   *
4   *
5   * File sorgente Mixer
6   * Francesco De Col - 2042803
7   * Scheda: NUCLEO-G431RB
8   *
9   *
10 */
11 /* USER CODE END Header */
12 /* Includes -----*/
13 #include "main.h"
14
15 /* Private includes -----*/
16 /* USER CODE BEGIN Includes */
17
18 /* USER CODE END Includes */
19
20 /* Private typedef -----*/
21 /* USER CODE BEGIN PTD */
22 typedef struct AudioCh {
23     uint16_t    xL[3], xR[3];
24     float       G_lp, G_bp, G_hp, G,
25                yL[3], yR[3],
26                zL_LP[3], zR_LP[3],
27                zL_BP[3], zR_BP[3],
28                zL_HP[3], zR_HP[3],
29                zL, zR;
30 } AudioCh;
```

```

31 /* USER CODE END PTD */
32
33 /* Private define -----*/
34 /* USER CODE BEGIN PD */
35 #define ADC_CONVERSION_NUMBER 7
36
37 // Coefficienti filtro passa basso
38 #define A0Inv 0.22371316367261522121f
39 #define A1 -7.97389856687315210943f
40 #define A2 3.55609178513218129241f
41 #define B0 0.01305071656342394528f
42 #define B1 0.02610143312684789056f
43 #define B2 0.01305071656342394528f
44
45 // Coefficienti filtro passa banda
46 #define C0Inv 0.22072150672837394881f
47 #define C1 -7.9669653736988331385f
48 #define C2 3.50243851529043544627f
49 #define D0 0.56548667764616278292f
50 #define D2 -0.56548667764616278292f
51
52 // Coefficienti filtro passa alto
53 #define E0Inv 0.21778946093655217558f
54 #define E1 -7.95921651073930017099f
55 #define E2 3.44919308034129659841f
56 #define F0 4.0f
57 #define F1 -8.0f
58 #define F2 4.0f
59
60 // Coefficienti filtro passa alto fisso
61 #define G0Inv 0.24996430390657009305f
62 #define G1 -7.9999999592165107393f
63 #define G2 3.99942882172745557703f
64 #define H0 4.0f
65 #define H1 -8.0f
66 #define H2 4.0f
67 /* USER CODE END PD */
68
69 /* Private macro -----*/
70 /* USER CODE BEGIN PM */
71
72 /* USER CODE END PM */
73
74 /* Private variables -----*/

```

```

75
76 COM_InitTypeDef BspCOMInit;
77 ADC_HandleTypeDef hadc1;
78 ADC_HandleTypeDef hadc2;
79 DMA_HandleTypeDef hdma_adc1;
80
81 DAC_HandleTypeDef hdac1;
82
83 TIM_HandleTypeDef htim6;
84
85 /* USER CODE BEGIN PV */
86 uint16_t      outL, outR;
87 uint32_t      adcDMABuffer[ADC_CONVERSION_NUMBER];
88 float         masterGain;
89 AudioCh       ch1, ch2;
90 /* USER CODE END PV */
91
92 /* Private function prototypes -----*/
93 void SystemClock_Config(void);
94 static void MX_GPIO_Init(void);
95 static void MX_DMA_Init(void);
96 static void MX_ADC1_Init(void);
97 static void MX_ADC2_Init(void);
98 static void MX_DAC1_Init(void);
99 static void MX_TIM6_Init(void);
100 /* USER CODE BEGIN PFP */
101 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc);
102 void AudioCh_DataUpdate(AudioCh* ch);
103 float AudioCh_SecondOrderLP(AudioCh* ch, float input[3], float output[3]);
104 float AudioCh_SecondOrderBP(AudioCh* ch, float input[3], float output[3]);
105 float AudioCh_SecondOrderHP(AudioCh* ch, float input[3], float output[3]);
106 float AudioCh_FixedSecondOrderHP(uint16_t input[3], float output[3]);
107 /* USER CODE END PFP */
108
109 /* Private user code -----*/
110 /* USER CODE BEGIN 0 */
111
112 /* USER CODE END 0 */
113
114 /**
115  * @brief The application entry point.
116  * @retval int
117  */
118 int main(void)

```

```

119 {
120
121     /* USER CODE BEGIN 1 */
122
123     /* USER CODE END 1 */
124
125     /* MCU Configuration-----*/
126
127     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
128     HAL_Init();
129
130     /* USER CODE BEGIN Init */
131
132     /* USER CODE END Init */
133
134     /* Configure the system clock */
135     SystemClock_Config();
136
137     /* USER CODE BEGIN SysInit */
138
139     /* USER CODE END SysInit */
140
141     /* Initialize all configured peripherals */
142     MX_GPIO_Init();
143     MX_DMA_Init();
144     MX_ADC1_Init();
145     MX_ADC2_Init();
146     MX_DAC1_Init();
147     MX_TIM6_Init();
148     /* USER CODE BEGIN 2 */
149     // Auto-calibrazione ADC
150     HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);
151     HAL_ADCEx_Calibration_Start(&hadc2, ADC_SINGLE_ENDED);
152
153     // Attivazione ADC in modalità duale
154     HAL_ADCEx_MultiModeStart_DMA(&hadc1, adcDMABuffer, ADC_CONVERSION_NUMBER);
155
156     // Attivazione DAC in modalità duale
157     HAL_DACEx_DualStart(&hdac1);
158
159     // Avvio timer
160     HAL_TIM_Base_Start(&htim6);
161     /* USER CODE END 2 */
162

```

```

163     /* Initialize USER push-button, will be used to trigger an interrupt each time it's
        pressed.*/
164     BSP_PB_Init(BUTTON_USER, BUTTON_MODE_EXTI);
165
166     /* Initialize COM1 port (115200, 8 bits (7-bit data + 1 stop bit), no parity */
167     BspCOMInit.BaudRate = 115200;
168     BspCOMInit.WordLength = COM_WORDLENGTH_8B;
169     BspCOMInit.StopBits = COM_STOPBITS_1;
170     BspCOMInit.Parity = COM_PARITY_NONE;
171     BspCOMInit.HwFlowCtl = COM_HWCONTROL_NONE;
172     if (BSP_COM_Init(COM1, &BspCOMInit) != BSP_ERROR_NONE)
173     {
174         Error_Handler();
175     }
176
177     /* Infinite loop */
178     /* USER CODE BEGIN WHILE */
179     while (1)
180     {
181         /* USER CODE END WHILE */
182
183         /* USER CODE BEGIN 3 */
184     }
185     /* USER CODE END 3 */
186 }
187
188 /**
189  * @brief System Clock Configuration
190  * @retval None
191  */
192 void SystemClock_Config(void)
193 {
194     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
195     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
196
197     /** Configure the main internal regulator output voltage
198     */
199     HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1_BOOST);
200
201     /** Initializes the RCC Oscillators according to the specified parameters
202     * in the RCC_OscInitTypeDef structure.
203     */
204     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
205     RCC_OscInitStruct.HSISState = RCC_HSI_ON;

```

```

206     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
207     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
208     RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
209     RCC_OscInitStruct.PLL.PLLM = RCC_PLLM_DIV4;
210     RCC_OscInitStruct.PLL.PLLN = 85;
211     RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
212     RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
213     RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
214     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
215     {
216         Error_Handler();
217     }
218
219     /** Initializes the CPU, AHB and APB buses clocks
220     */
221     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSClk
222     |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
223     RCC_ClkInitStruct.SYSClkSource = RCC_SYSClkSOURCE_PLLCLK;
224     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSClk_DIV1;
225     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
226     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
227
228     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK)
229     {
230         Error_Handler();
231     }
232 }
233
234 /**
235  * @brief ADC1 Initialization Function
236  * @param None
237  * @retval None
238  */
239 static void MX_ADC1_Init(void)
240 {
241
242     /* USER CODE BEGIN ADC1_Init 0 */
243
244     /* USER CODE END ADC1_Init 0 */
245
246     ADC_MultiModeTypeDef multimode = {0};
247     ADC_ChannelConfTypeDef sConfig = {0};
248
249     /* USER CODE BEGIN ADC1_Init 1 */

```

```

250
251  /* USER CODE END ADC1_Init 1 */
252
253  /** Common config
254  */
255  hadc1.Instance = ADC1;
256  hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
257  hadc1.Init.Resolution = ADC_RESOLUTION_12B;
258  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
259  hadc1.Init.GainCompensation = 0;
260  hadc1.Init.ScanConvMode = ADC_SCAN_ENABLE;
261  hadc1.Init.EOCSelection = ADC_EOC_SEQ_CONV;
262  hadc1.Init.LowPowerAutoWait = DISABLE;
263  hadc1.Init.ContinuousConvMode = DISABLE;
264  hadc1.Init.NbrOfConversion = 7;
265  hadc1.Init.DiscontinuousConvMode = DISABLE;
266  hadc1.Init.ExternalTrigConv = ADC_EXTERNALTRIG_T6_TRGO;
267  hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_RISING;
268  hadc1.Init.DMAContinuousRequests = ENABLE;
269  hadc1.Init.Overrun = ADC_OVR_DATA_PRESERVED;
270  hadc1.Init.OversamplingMode = DISABLE;
271  if (HAL_ADC_Init(&hadc1) != HAL_OK)
272  {
273      Error_Handler();
274  }
275
276  /** Configure the ADC multi-mode
277  */
278  multimode.Mode = ADC_DUALMODE_REGSIMULT;
279  multimode.DMAAccessMode = ADC_DMAACCESSMODE_12_10_BITS;
280  multimode.TwoSamplingDelay = ADC_TWOSAMPLINGDELAY_1CYCLE;
281  if (HAL_ADCEx_MultiModeConfigChannel(&hadc1, &multimode) != HAL_OK)
282  {
283      Error_Handler();
284  }
285
286  /** Configure Regular Channel
287  */
288  sConfig.Channel = ADC_CHANNEL_1;
289  sConfig.Rank = ADC_REGULAR_RANK_1;
290  sConfig.SamplingTime = ADC_SAMPLETIME_12CYCLES_5;
291  sConfig.SingleDiff = ADC_SINGLE_ENDED;
292  sConfig.OffsetNumber = ADC_OFFSET_NONE;
293  sConfig.Offset = 0;

```

```

294     if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
295     {
296         Error_Handler();
297     }
298
299     /** Configure Regular Channel
300     */
301     sConfig.Channel = ADC_CHANNEL_2;
302     sConfig.Rank = ADC_REGULAR_RANK_2;
303     if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
304     {
305         Error_Handler();
306     }
307
308     /** Configure Regular Channel
309     */
310     sConfig.Channel = ADC_CHANNEL_5;
311     sConfig.Rank = ADC_REGULAR_RANK_3;
312     if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
313     {
314         Error_Handler();
315     }
316
317     /** Configure Regular Channel
318     */
319     sConfig.Channel = ADC_CHANNEL_7;
320     sConfig.Rank = ADC_REGULAR_RANK_4;
321     if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
322     {
323         Error_Handler();
324     }
325
326     /** Configure Regular Channel
327     */
328     sConfig.Channel = ADC_CHANNEL_6;
329     sConfig.Rank = ADC_REGULAR_RANK_5;
330     if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
331     {
332         Error_Handler();
333     }
334
335     /** Configure Regular Channel
336     */
337     sConfig.Channel = ADC_CHANNEL_11;

```



```

338     sConfig.Rank = ADC_REGULAR_RANK_6;
339     if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
340     {
341         Error_Handler();
342     }
343
344     /** Configure Regular Channel
345     */
346     sConfig.Channel = ADC_CHANNEL_12;
347     sConfig.Rank = ADC_REGULAR_RANK_7;
348     if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
349     {
350         Error_Handler();
351     }
352     /* USER CODE BEGIN ADC1_Init 2 */
353
354     /* USER CODE END ADC1_Init 2 */
355
356 }
357
358 /**
359  * @brief ADC2 Initialization Function
360  * @param None
361  * @retval None
362  */
363 static void MX_ADC2_Init(void)
364 {
365
366     /* USER CODE BEGIN ADC2_Init 0 */
367
368     /* USER CODE END ADC2_Init 0 */
369
370     ADC_ChannelConfTypeDef sConfig = {0};
371
372     /* USER CODE BEGIN ADC2_Init 1 */
373
374     /* USER CODE END ADC2_Init 1 */
375
376     /** Common config
377     */
378     hadc2.Instance = ADC2;
379     hadc2.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
380     hadc2.Init.Resolution = ADC_RESOLUTION_12B;
381     hadc2.Init.DataAlign = ADC_DATAALIGN_RIGHT;

```

```

382     hadc2.Init.GainCompensation = 0;
383     hadc2.Init.ScanConvMode = ADC_SCAN_ENABLE;
384     hadc2.Init.EOCSelection = ADC_EOC_SEQ_CONV;
385     hadc2.Init.LowPowerAutoWait = DISABLE;
386     hadc2.Init.ContinuousConvMode = DISABLE;
387     hadc2.Init.NbrOfConversion = 7;
388     hadc2.Init.DiscontinuousConvMode = DISABLE;
389     hadc2.Init.DMAContinuousRequests = DISABLE;
390     hadc2.Init.Overrun = ADC_OVR_DATA_PRESERVED;
391     hadc2.Init.OversamplingMode = DISABLE;
392     if (HAL_ADC_Init(&hadc2) != HAL_OK)
393     {
394         Error_Handler();
395     }
396
397     /** Configure Regular Channel
398     */
399     sConfig.Channel = ADC_CHANNEL_3;
400     sConfig.Rank = ADC_REGULAR_RANK_1;
401     sConfig.SamplingTime = ADC_SAMPLETIME_12CYCLES_5;
402     sConfig.SingleDiff = ADC_SINGLE_ENDED;
403     sConfig.OffsetNumber = ADC_OFFSET_NONE;
404     sConfig.Offset = 0;
405     if (HAL_ADC_ConfigChannel(&hadc2, &sConfig) != HAL_OK)
406     {
407         Error_Handler();
408     }
409
410     /** Configure Regular Channel
411     */
412     sConfig.Channel = ADC_CHANNEL_4;
413     sConfig.Rank = ADC_REGULAR_RANK_2;
414     if (HAL_ADC_ConfigChannel(&hadc2, &sConfig) != HAL_OK)
415     {
416         Error_Handler();
417     }
418
419     /** Configure Regular Channel
420     */
421     sConfig.Channel = ADC_CHANNEL_5;
422     sConfig.Rank = ADC_REGULAR_RANK_3;
423     if (HAL_ADC_ConfigChannel(&hadc2, &sConfig) != HAL_OK)
424     {
425         Error_Handler();

```

```

426     }
427
428     /** Configure Regular Channel
429     */
430     sConfig.Channel = ADC_CHANNEL_9;
431     sConfig.Rank = ADC_REGULAR_RANK_4;
432     if (HAL_ADC_ConfigChannel(&hadc2, &sConfig) != HAL_OK)
433     {
434         Error_Handler();
435     }
436
437     /** Configure Regular Channel
438     */
439     sConfig.Channel = ADC_CHANNEL_8;
440     sConfig.Rank = ADC_REGULAR_RANK_5;
441     if (HAL_ADC_ConfigChannel(&hadc2, &sConfig) != HAL_OK)
442     {
443         Error_Handler();
444     }
445
446     /** Configure Regular Channel
447     */
448     sConfig.Channel = ADC_CHANNEL_11;
449     sConfig.Rank = ADC_REGULAR_RANK_6;
450     if (HAL_ADC_ConfigChannel(&hadc2, &sConfig) != HAL_OK)
451     {
452         Error_Handler();
453     }
454
455     /** Configure Regular Channel
456     */
457     sConfig.Rank = ADC_REGULAR_RANK_7;
458     if (HAL_ADC_ConfigChannel(&hadc2, &sConfig) != HAL_OK)
459     {
460         Error_Handler();
461     }
462     /* USER CODE BEGIN ADC2_Init 2 */
463
464     /* USER CODE END ADC2_Init 2 */
465
466 }
467
468 /**
469  * @brief DAC1 Initialization Function

```

```

470 * @param None
471 * @retval None
472 */
473 static void MX_DAC1_Init(void)
474 {
475
476     /* USER CODE BEGIN DAC1_Init 0 */
477
478     /* USER CODE END DAC1_Init 0 */
479
480     DAC_ChannelConfTypeDef sConfig = {0};
481
482     /* USER CODE BEGIN DAC1_Init 1 */
483
484     /* USER CODE END DAC1_Init 1 */
485
486     /** DAC Initialization
487     */
488     hdac1.Instance = DAC1;
489     if (HAL_DAC_Init(&hdac1) != HAL_OK)
490     {
491         Error_Handler();
492     }
493
494     /** DAC channel OUT1 config
495     */
496     sConfig.DAC_HighFrequency = DAC_HIGH_FREQUENCY_INTERFACE_MODE_AUTOMATIC;
497     sConfig.DAC_DMADoubleDataMode = DISABLE;
498     sConfig.DAC_SignedFormat = DISABLE;
499     sConfig.DAC_SampleAndHold = DAC_SAMPLEANDHOLD_DISABLE;
500     sConfig.DAC_Trigger = DAC_TRIGGER_T6_TRGO;
501     sConfig.DAC_Trigger2 = DAC_TRIGGER_NONE;
502     sConfig.DAC_OutputBuffer = DAC_OUTPUTBUFFER_ENABLE;
503     sConfig.DAC_ConnectOnChipPeripheral = DAC_CHIPCONNECT_EXTERNAL;
504     sConfig.DAC_UserTrimming = DAC_TRIMMING_FACTORY;
505     if (HAL_DAC_ConfigChannel(&hdac1, &sConfig, DAC_CHANNEL_1) != HAL_OK)
506     {
507         Error_Handler();
508     }
509
510     /** DAC channel OUT2 config
511     */
512     if (HAL_DAC_ConfigChannel(&hdac1, &sConfig, DAC_CHANNEL_2) != HAL_OK)
513     {

```

```

514     Error_Handler();
515 }
516 /* USER CODE BEGIN DAC1_Init 2 */
517
518 /* USER CODE END DAC1_Init 2 */
519
520 }
521
522 /**
523  * @brief TIM6 Initialization Function
524  * @param None
525  * @retval None
526  */
527 static void MX_TIM6_Init(void)
528 {
529
530     /* USER CODE BEGIN TIM6_Init 0 */
531
532     /* USER CODE END TIM6_Init 0 */
533
534     TIM_MasterConfigTypeDef sMasterConfig = {0};
535
536     /* USER CODE BEGIN TIM6_Init 1 */
537
538     /* USER CODE END TIM6_Init 1 */
539     htim6.Instance = TIM6;
540     htim6.Init.Prescaler = 170-1;
541     htim6.Init.CounterMode = TIM_COUNTERMODE_UP;
542     htim6.Init.Period = 24;
543     htim6.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
544     if (HAL_TIM_Base_Init(&htim6) != HAL_OK)
545     {
546         Error_Handler();
547     }
548     sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
549     sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
550     if (HAL_TIMEx_MasterConfigSynchronization(&htim6, &sMasterConfig) != HAL_OK)
551     {
552         Error_Handler();
553     }
554     /* USER CODE BEGIN TIM6_Init 2 */
555
556     /* USER CODE END TIM6_Init 2 */
557

```

```

558 }
559
560 /**
561  * Enable DMA controller clock
562  */
563 static void MX_DMA_Init(void)
564 {
565
566     /* DMA controller clock enable */
567     __HAL_RCC_DMAMUX1_CLK_ENABLE();
568     __HAL_RCC_DMA1_CLK_ENABLE();
569
570     /* DMA interrupt init */
571     /* DMA1_Channel1_IRQn interrupt configuration */
572     HAL_NVIC_SetPriority(DMA1_Channel1_IRQn, 0, 0);
573     HAL_NVIC_EnableIRQ(DMA1_Channel1_IRQn);
574
575 }
576
577 /**
578  * @brief GPIO Initialization Function
579  * @param None
580  * @retval None
581  */
582 static void MX_GPIO_Init(void)
583 {
584     /* USER CODE BEGIN MX_GPIO_Init_1 */
585
586     /* USER CODE END MX_GPIO_Init_1 */
587
588     /* GPIO Ports Clock Enable */
589     __HAL_RCC_GPIOC_CLK_ENABLE();
590     __HAL_RCC_GPIOF_CLK_ENABLE();
591     __HAL_RCC_GPIOA_CLK_ENABLE();
592     __HAL_RCC_GPIOB_CLK_ENABLE();
593
594     /* USER CODE BEGIN MX_GPIO_Init_2 */
595
596     /* USER CODE END MX_GPIO_Init_2 */
597 }
598
599 /* USER CODE BEGIN 4 */
600 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
601     // Aggiornamento dati filtri, estrazione e calcolo nuovi campioni

```

```

602     AudioCh_DataUpdate(&ch1);
603     AudioCh_DataUpdate(&ch2);
604
605     // Calcolo guadagno master
606     masterGain = ((uint16_t)adcDMABuffer[6]) * 0.0002442002442002442f;
607     // Somma segnali filtrati e aggiunta offset DC per DAC
608     outL = ((&ch1)->zL + (&ch2)->zL) * masterGain + 2048;
609     outR = ((&ch1)->zR + (&ch2)->zR) * masterGain + 2048;
610
611     // Check limiti valori in uscita dal DAC
612     if (outL > 4095) outL = 4095;
613     else if (outL < 0) outL = 0;
614     else if (outR > 4095) outR = 4095;
615     else if (outR < 0) outR = 0;
616
617     // Set nuovi campioni uscita DAC
618     HAL_DACEx_DualSetValue(&hdac1, DAC_ALIGN_12B_R, outL, outR);
619 }
620
621 void AudioCh_DataUpdate(AudioCh* ch) {
622     // Aggiornamento campioni I/O filtri lato sinistro
623     ch->xL[2] = ch->xL[1];
624     ch->xL[1] = ch->xL[0];
625     ch->yL[2] = ch->yL[1];
626     ch->yL[1] = ch->yL[0];
627     ch->zL_LP[2] = ch->zL_LP[1];
628     ch->zL_LP[1] = ch->zL_LP[0];
629     ch->zL_BP[2] = ch->zL_BP[1];
630     ch->zL_BP[1] = ch->zL_BP[0];
631     ch->zL_HP[2] = ch->zL_HP[1];
632     ch->zL_HP[1] = ch->zL_HP[0];
633     // Aggiornamento campioni I/O filtri lato destro
634     ch->xR[2] = ch->xR[1];
635     ch->xR[1] = ch->xR[0];
636     ch->yR[2] = ch->yR[1];
637     ch->yR[1] = ch->yR[0];
638     ch->zR_LP[2] = ch->zR_LP[1];
639     ch->zR_LP[1] = ch->zR_LP[0];
640     ch->zR_BP[2] = ch->zR_BP[1];
641     ch->zR_BP[1] = ch->zR_BP[0];
642     ch->zR_HP[2] = ch->zR_HP[1];
643     ch->zR_HP[1] = ch->zR_HP[0];
644
645     // Calcolo guadagni ed estrazione nuovi valori campionati

```

```

646     int i = 0;
647     if (ch == &ch1) i = 0;
648     else if (ch == &ch2) i = 1;
649     ch->xL[0] = adcDMABuffer[i] >> 16;
650     ch->xR[0] = (uint16_t)adcDMABuffer[i];
651     ch->G_hp = ((adcDMABuffer[i+2] >> 16) < 2048) ? (0.5f + 0.00024425989252564728f * (
        adcDMABuffer[i+2] >> 16)) : (0.0004885197850512945f * ((adcDMABuffer[i+2] >>
        16) - 1));
652     ch->G_bp = ((uint16_t)adcDMABuffer[i+2] < 2048) ? (0.5f + 0.00024425989252564728f *
        (uint16_t)adcDMABuffer[i+2]) : 0.0004885197850512945f * ((uint16_t)
        adcDMABuffer[i+2] - 1);
653     ch->G_lp = ((adcDMABuffer[i+4] >> 16) < 2048) ? (0.5f + 0.00024425989252564728f * (
        adcDMABuffer[i+4] >> 16)) : 0.0004885197850512945f * ((adcDMABuffer[i+4] >> 16)
        - 1);
654     ch->G = ((uint16_t)adcDMABuffer[i+4]) * 0.0002442002442002442f;
655
656     // Filtraggio segnali digitali
657     ch->yL[0] = AudioCh_FixedSecondOrderHP(ch->xL, ch->yL);
658     ch->yR[0] = AudioCh_FixedSecondOrderHP(ch->xR, ch->yR);
659     ch->zL_LP[0] = AudioCh_SecondOrderLP(ch, ch->yL, ch->zL_LP);
660     ch->zL_BP[0] = AudioCh_SecondOrderBP(ch, ch->yL, ch->zL_BP);
661     ch->zL_HP[0] = AudioCh_SecondOrderHP(ch, ch->yL, ch->zL_HP);
662     ch->zL = ch->G * (ch->zL_LP[0] + ch->zL_BP[0] + ch->zL_HP[0]);
663     ch->zR_LP[0] = AudioCh_SecondOrderLP(ch, ch->yR, ch->zR_LP);
664     ch->zR_BP[0] = AudioCh_SecondOrderBP(ch, ch->yR, ch->zR_BP);
665     ch->zR_HP[0] = AudioCh_SecondOrderHP(ch, ch->yR, ch->zR_HP);
666     ch->zR = ch->G * (ch->zR_LP[0] + ch->zR_BP[0] + ch->zR_HP[0]);
667 }
668
669 float AudioCh_SecondOrderLP(AudioCh* ch, float input[3], float output[3]) {
670     return AOInv * ((ch->G_lp)*B0*input[0] + (ch->G_lp)*B1*input[1] + (ch->G_lp)*B2*
        input[2] - A1*output[1] - A2*output[2]);
671 }
672
673 float AudioCh_SecondOrderBP(AudioCh* ch, float input[3], float output[3]) {
674     return COInv * ((ch->G_bp)*D0*input[0] + (ch->G_bp)*D2*input[2] - C1*output[1] - C2
        *output[2]);
675 }
676
677 float AudioCh_SecondOrderHP(AudioCh* ch, float input[3], float output[3]) {
678     return EOInv * ((ch->G_hp)*F0*input[0] + (ch->G_hp)*F1*input[1] + (ch->G_hp)*F2*
        input[2] - E1*output[1] - E2*output[2]);
679 }
680

```



```

681 float AudioCh_FixedSecondOrderHP(uint16_t input[3], float output[3]) {
682     return G0Inv * (H0*input[0] + H1*input[1] + H2*input[2] - G1*output[1] - G2*output
        [2]);
683 }
684 /* USER CODE END 4 */
685
686 /**
687  * @brief This function is executed in case of error occurrence.
688  * @retval None
689  */
690 void Error_Handler(void)
691 {
692     /* USER CODE BEGIN Error_Handler_Debug */
693     /* User can add his own implementation to report the HAL error return state */
694     __disable_irq();
695     while (1)
696     {
697     }
698     /* USER CODE END Error_Handler_Debug */
699 }
700 #ifndef USE_FULL_ASSERT
701 /**
702  * @brief Reports the name of the source file and the source line number
703  *         where the assert_param error has occurred.
704  * @param file: pointer to the source file name
705  * @param line: assert_param error line source number
706  * @retval None
707  */
708 void assert_failed(uint8_t *file, uint32_t line)
709 {
710     /* USER CODE BEGIN 6 */
711     /* User can add his own implementation to report the file name and line number,
712     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
713     /* USER CODE END 6 */
714 }
715 #endif /* USE_FULL_ASSERT */

```

Listato A.1: File main.c completo



# Bibliografia

- [1] Wikipedia contributors, *Hearing range* — *Wikipedia, The Free Encyclopedia*, [https://en.wikipedia.org/w/index.php?title=Hearing\\_range&oldid=1292107613](https://en.wikipedia.org/w/index.php?title=Hearing_range&oldid=1292107613), [Online; accesso eseguito il: 27/08/2025], 2025.
- [2] Wikipedia contributors, *Nyquist–Shannon sampling theorem* — *Wikipedia, The Free Encyclopedia*, [https://en.wikipedia.org/w/index.php?title=Nyquist%E2%80%93Shannon\\_sampling\\_theorem&oldid=1296907122](https://en.wikipedia.org/w/index.php?title=Nyquist%E2%80%93Shannon_sampling_theorem&oldid=1296907122), [Online; accesso eseguito il: 14/08/2025], 2025.
- [3] STMicroelectronics, *STM32 Nucleo boards*, <https://www.st.com/en/evaluation-tools/stm32-nucleo-boards.html>, [Online; accesso eseguito il: 13/08/2025].
- [4] STMicroelectronics, *NUCLEO-G431RB Overview*, <https://www.st.com/en/evaluation-tools/nucleo-g431rb.html>, [Online; accesso eseguito il: 26/08/2025].
- [5] STMicroelectronics, *DB2196 - STM32 Nucleo-64 boards*, [https://www.st.com/resource/en/data\\_brief/nucleo-g431rb.pdf](https://www.st.com/resource/en/data_brief/nucleo-g431rb.pdf), [Online; accesso eseguito il: 13/08/2025].
- [6] STMicroelectronics, *DS12589 - Arm® Cortex®-M4 32-bit MCU+FPU, 170 MHz /213 DMIPS, up to 128 KB Flash, 32 KB SRAM, rich analog, math accelerator*, <https://www.st.com/resource/en/datasheet/stm32g431rb.pdf>, [Online; accesso eseguito il: 13/08/2025].
- [7] STMicroelectronics, *RM0440 - STM32G4 series advanced Arm®-based 32-bit MCUs*, [https://www.st.com/resource/en/reference\\_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf](https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf), [Online; accesso eseguito il: 13/08/2025].
- [8] Microchip Technology Inc., *MCP6001/1R/1U/2/4 - Data Sheet*, <https://ww1.microchip.com/downloads/aemDocuments/documents/MSLD/ProductDocuments/DataSheets/MCP6001-1R-1U-2-4-1-MHz-Low-Power-Op-Amp-DS20001733L.pdf>, [Online; accesso eseguito il: 12/08/2025].

- [9] Wikipedia contributors, *Infinite impulse response* — *Wikipedia, The Free Encyclopedia*, [https://en.wikipedia.org/w/index.php?title=Infinite\\_impulse\\_response&oldid=1298363603](https://en.wikipedia.org/w/index.php?title=Infinite_impulse_response&oldid=1298363603), [Online; accesso eseguito il: 13/08/2025], 2025.
- [10] Wikipedia contributors, *Z-transform* — *Wikipedia, The Free Encyclopedia*, <https://en.wikipedia.org/w/index.php?title=Z-transform&oldid=1302803780>, [Online; accesso eseguito il: 13/08/2025], 2025.
- [11] Wikipedia contributors, *Bilinear transform* — *Wikipedia, The Free Encyclopedia*, [https://en.wikipedia.org/w/index.php?title=Bilinear\\_transform&oldid=1286084688](https://en.wikipedia.org/w/index.php?title=Bilinear_transform&oldid=1286084688), [Online; accesso eseguito il: 13/08/2025], 2025.
- [12] STMicroelectronics, *DB3871 - Integrated development environment for STM32 products*, [https://www.st.com/resource/en/data\\_brief/stm32cubeide.pdf](https://www.st.com/resource/en/data_brief/stm32cubeide.pdf), [Online; accesso eseguito il: 12/08/2025].
- [13] Wikipedia contributors, *Compact Disc Digital Audio* — *Wikipedia, The Free Encyclopedia*, [https://en.wikipedia.org/w/index.php?title=Compact\\_Disc\\_Digital\\_Audio&oldid=1310423474](https://en.wikipedia.org/w/index.php?title=Compact_Disc_Digital_Audio&oldid=1310423474), [Online; accesso eseguito il: 11/09/2025], 2025.